Journal of Intelligent & Robotic Systems (2025) 111:78
https://doi.org/10.1007/s10846-024-02212-1

REGULAR PAPER

# Yara: An Ocean Virtual Environment for Research and Development of Autonomous Sailing Robots and Other Unmanned Surface Vessels

**Eduardo Charles Vasconcellos[1] · Álvaro Pinto Fernandes Negreiros[2] · André Paulo Dantas de Araújo[1,4] · Raphael Guerra[4] · Philippe Preux[1] · Davi Henrique dos Santos[2,3] · Luiz Marcos Garcia Gonçalves[2] · Esteban Walter Gonzalez Clua[4]**

Received: 4 June 2023 / Accepted: 11 December 2024 / Published online: 5 July 2025
© The Author(s) 2025

**Abstract**

Overall, a big challenge in building a sailboat USV relies on the development of an autonomous system for guidance, navigation, and control (GNC) because both sail and rudder angle must be cooperatively adjusted to correct the navigation direction — traditional propelled boats can be more easily controlled with a straightforward control task to set the rudder angle. Moreover, sailing upwind requires special maneuvers to reach a given target in that unfeasible direction. Reinforcement learning emerges as a promising technique for building autonomous GNCs for sailing robots, but training the neural network with a real sailboat is impractical due to long periods of training and safety reasons. Even traditional control-based approaches are mainly tested in simulated environments due to the difficulties in building and operating a real sailboat. The issue that arises is the fidelity of these simulated environments. In this context, we propose Yara, an oceanic virtual environment with a reliable physics simulation for developing, training, and evaluating autonomous agents to operate digital twins of sailing robots in reinforcement learning and other paradigms. An autonomous sailing robot digital twin is available within the virtual environment, with the foil dynamics constructed based on a real sailing robot. We coupled these foil dynamics in Gazebo's physics engine to compute the lift and drag forces acting on the sail, rudder, and keel. The simulated world feeds sensors such as cameras, wind sensors, and GPS. The Robot Operating System communicates these sensors' data through topics, facilitating users' implementation and testing of new GNC solutions. Yara provides a reliable solution for foil dynamic simulated physics that achieves a simulation speedup of 300 times on an i7 laptop with 8 GB of RAM, powered by a Nvidia RTX 3060 and running Ubuntu 20.04. With this speedup, it is possible to complete a million time steps of deep reinforcement learning training in approximately eight hours. Evaluation scenarios were presented to highlight specific features of the simulator, like the maneuverability of the sailing robot digital twin and applications to train, evaluate, and compare reinforcement learning agents and other control solutions.

**Keywords** Digital twin · Unmanned surface vessel · Autonomous sailboat · Reinforcement learning

## 1 Introduction

The development of Unmanned Surface Vessels (USVs), also known as Autonomous Surface Vessels (ASVs), is currently an interesting and exciting research field, with a vast amount of applications such as surveillance, rescue missions, structural inspection, and environmental monitoring, among others. As a USV can be operated remotely or autonomously, it can perform dangerous and/or long-duration missions at a cost smaller than a manned vessel. In this work, we deal with a sailing robot, a particular type of USV where the main source of propulsion is the wind. Developing and assembling such a project can be challenging from both perspectives, hardware and software [1]. This type of vessel has some advantages when dealing with specific applications, such as its low cost and self-sustainability. As its main propulsion force comes from the wind, a sailing robot performs long missions consuming less energy than other USVs [2, 3]. Works like the ones of Ang et al. [4] and Negreiros et al. [3] advocate sailing robots as a reliable instrument for different tasks in environmental monitoring.

Mendonça et al. [5], Plumet et al. [6], and Jing et al. [7] discuss important issues that arise when developing and testing autonomous systems. One of the main concerns is that any

Extended author information available on the last page of the article

《智能与 Robotic Systems 杂志》(2025) 111:78 https://
doi.org/10.1007/s10846-024-02212-1

常规论文

# Yara：用于自主航行机器人及其他无人水面艇研发的海洋虚拟环境

爱德华多·查尔斯·巴斯孔塞洛斯[1] · 阿尔瓦罗·平托·费尔南德斯·内格雷罗斯[2] · 安德烈·保罗·丹塔斯·德·阿劳霍[1,4] · 拉斐尔·格拉[4] · 菲利普·普雷[1] · 达维·恩里克·多斯·桑托斯[2,3] · 路易斯·马科斯·加西亚·贡萨尔维斯[2] · 埃斯特班·瓦尔特·冈萨雷斯·克卢阿[4]

接收日期：2023年6月4日/接受日期：2024年12月11日/在线发表日期：2025年7月5日
© 作者 2025

**摘要**

总体而言，建造帆船型无人水面艇（USV）面临的主要挑战在于开发用于制导、导航与控制（GNC）的自主系统，因为必须协同调整帆面与舵角以修正航行方向——传统推进艇只需通过简单舵角设定即可实现更便捷的操控。此外，迎风航行需要特殊机动才能抵达特定目标区域，而该方向存在实际操作难度。强化学习技术为构建帆船机器人自主航向控制系统（GNC）展现出巨大潜力，但由于训练周期过长及安全考量，使用真实帆船进行神经网络训练并不现实。传统控制方法也因建造与操作真实帆船的困难，主要在模拟环境中进行测试。由此产生的核心问题在于模拟环境的逼真度。基于此，我们提出Yara海洋虚拟环境方案，该方案通过可靠的物理仿真技术，为开发、训练和评估自主智能体提供支持，使其能在强化学习等范式中操控帆船机器人的数字孪生体。虚拟环境中可使用自主航行机器人数字孪生模型，其翼型动力学特性基于真实航行机器人构建。我们将这些翼型动力学参数与Gazebo物理引擎进行耦合计算，以获取作用于帆面、舵面及龙骨的升力与阻力。模拟环境实时采集摄像头、风速传感器及GPS等设备数据，机器人操作系统通过主题通信机制传输传感器数据，为用户实现新 GNC 方案的部署与测试提供支持。Yara平台为翼型动力学模拟物理计算提供可靠解决方案，在搭载Nvidia RTX 3060显卡、运行Ubuntu 20.04系统的i7笔记本电脑（8GB内存）上，可实现300倍的模拟速度提升。基于此性能优势，深度强化学习训练百万时间步长仅需约八小时即可完成。通过展示评估场景，重点突显了该模拟器的核心特性：包括航行机器人数字孪生的机动性表现，以及在强化学习智能体训练、性能评估与控制方案对比中的应用价值。

**关键词**数字孪生 · 无人水面舰艇 · 自主帆船 · 强化学习

## 1 引言

无人水面舰艇（USV，亦称自主水面舰艇ASV）的研发领域当前正蓬勃发展，其应用场景涵盖监控、救援行动、结构检测及环境监测等广泛领域。由于这类舰艇可实现远程操控或自主航行，相较于载人舰艇，其执行高危任务或长期任务的成本优势显著。本研究聚焦于帆船型机器人——作为特殊类型的无人水面舰艇，其核心动力系统...

推进系统中风力作为主要动力源的相关扩展作者信息详见文章末页。从硬件和软件两个维度来看，这类项目的研发与组装都面临诸多挑战[1]。不过在特定应用场景中，这类船舶具有成本低廉、可自主续航等优势。由于主要推进力源自风能，帆船机器人执行长途任务时能耗显著低于其他水下航行器[2, 3]。安格团队等学者的研究成果便是典型例证。[4] 以及Negreiros等[3] 倡导将航行机器人作为环境监测中多种任务的可靠工具。

门多卡酯[5]Plumetetal.[6]以及Jing等人[7]探讨在开发和测试自主系统过程中出现的重要问题。主要关注点之一是任何

unintended behavior can lead to severe damage to the vessel, the environment, and possibly cause injuries. Another significant challenge is the time-consuming nature of system development, particularly in the case of reinforcement learning (RL) applications where the agent interacts with the environment to create a policy to support its guidance, navigation, and control (GNC) decision, as stated by Lapan et al. [8].

To alleviate these issues, virtual environments and digital twins have been used as safe environments to develop and test autonomous vessels . Virtual environments not only offer a safe method for testing autonomous systems, but they also allow for the simultaneous evaluation of multiple solutions, quick and easy re-running of experiments, control over environmental conditions, and the simulation of rare real-world events [7]. However, creating a simulator for sailing robots always involves simulating certain aspects of physics, which are not typically included in the physics engines used by simulation tools. Generally, the physics engines used in robotic and vehicle simulations are built to calculate the dynamics of rigid bodies under the influence of gravitational force. However, in order to simulate a sailing boat accurately, it's necessary to account for hydrodynamic, hydrostatic, and aerodynamic forces.

To address the aforementioned issues, we introduce Yara, a new and effective ocean virtual environment (OVE) for the development, training (using RL), and assessment of autonomous GNC systems for sailing robots. Our OVE is open source and has been created within the Gazebo simulator [9], providing a high-quality 3D scene and accurate physics to simulate the maneuverability of sailing boats and their response to various wind conditions. In order to accurately simulate the responses of the sail, rudder, and keel to various environmental conditions, we have integrated the E-Boat, an actual sailing robot digital twin, into Yara. The dynamics of the foils have been added to Gazebo's physics engine to calculate the lift and drag forces on the sail, rudder, and keel. Yara simulates various sensors, including cameras, wind sensors, and GPS, as well as the realistic behavior of actuators that control the sail, rudder, and electric propeller. Sensor data and actuator commands are communicated through Robot Operating System (ROS) topics, which makes it simple for ROS developers to integrate and test new guidance, navigation, and control (GNC) solutions. We integrate our platform with Stable-baselines3 [10] using the Gymnasium [11] API to train reinforcement learning agents for GNC tasks.

Thus, the main contribution of our work is the USV simulator itself, which has in addition to traditional simulators the following novel and/or improved characteristics:

- wind variability extrapolated from meteorological data from a real large aquatic ecosystem in Brazil (the Guanabara Bay);
- wave pattern coupled to the wind direction and speed;
- a sailing robot digital twin modeled in a way its simulated dynamic behavior in response to different environmental conditions (such as wind, waves, and water currents) and actuator commands reflects the actual behavior of a real sailing robot.
- Python libraries that enable ready-to-go RL training using Gymnasium and Stable-baselines 3 APIs.
- a GitHub repository containing all Yara codes, physics models, scenarios, obstacles and the sail robot digital twin (https://github.com/medialab-fboat/Yara_OVE.git)

This document is organized as described in the following. Section 2 reviews state-of-the-art autonomous sailing systems and virtual environments for marine robotics. Section 3 presents the theory and functioning of some simulators used in autonomous vessel research. In Section 4, we propose our virtual sailing environment, introducing it with all its details. In Section 5, we introduce and discuss the responses of our virtual sailing robot in some maneuvers tests and depict the performance of three solutions to control the sailing robot through a seven-waypoints mission. Finally, in Section 6, we draw our conclusions and future research lines on this project.

## 2 Related Works

This section brings a careful review of the state-of-the-art, both on autonomous sailing systems (Section 2.1) and on virtual environments for marine robotics (Section 2.2).

### 2.1 Autonomous Sailing

It is not the intention of this section to do a deep review and discussion of autonomous GNC for sailing boats. However, this topic is relevant in the context of our proposed simulation tool.

Creating an autonomous system for guidance, navigation, and control (GNC) poses a significant challenge in building an unmanned sailboat, as discussed by Plumet et al. [12], Plumet et al. [6], Shen et al. [13], Silva Júnior et al. [14], and Shen et al. [15], among others. Steering and controlling a sailing robot involve much more intricate maneuvers than a motor boat. The aerodynamic forces acting on the sail are the main propeller of a sailing robot, and the poor management of these forces could reduce the robot's performance and pose

非预期行为可能导致船舶及环境遭受严重损害，并可能造成人员伤亡。另一个重大挑战在于系统开发耗时较长，特别是在强化学习（RL）应用中——正如Lapan等人所述，智能体需通过与环境交互来制定策略，以支持其航向控制、导航及控制（GNC）决策。[8].

为解决这些问题，虚拟环境与数字孪生技术已被用作开发和测试自主船舶的安全环境。虚拟环境不仅为自主系统测试提供了安全方案，还能实现多方案同步评估、实验快速复现、环境条件精准调控以及罕见真实场景模拟[7]。但需注意的是，为航行机器人构建仿真器时，始终需要模拟某些物理特性——这些内容通常未被包含在仿真工具所使用的物理引擎中。一般而言，机器人与车辆仿真所用物理引擎主要针对刚体在重力作用下的动力学计算进行设计。然而要实现对帆船的精确模拟，必须综合考虑流体动力、静水压力及空气动力等多重作用力。

为解决上述问题，我们推出了Yara——一款新型高效的海洋虚拟环境（OVE），专为帆船机器人自主GNC 系统的开发、训练（采用强化学习技术）及性能评估而设计。该虚拟环境采用开源架构，基于Gazebo仿真器[9]构建，通过高质量三维场景与精准物理模型，真实模拟帆船操控特性及其对不同风况的响应。为精确模拟帆面、舵机与龙骨在各类环境条件下的动态响应，我们整合了真实帆船机器人数字孪生体E-Boat至Yara系统。通过将翼型动力学模型集成至Gazebo物理引擎，可计算帆面、舵机及龙骨所受升力与阻力。Yara完整模拟了摄像头、风速传感器、GPS等各类传感器数据，并真实还原控制帆面、舵机及电动螺旋桨的执行器行为。传感器数据与执行器指令通过机器人操作系统（ROS）话题进行交互传输，使ROS开发者能便捷集成并测试新型制导、导航与控制（GNC）解决方案。我们通过 Gymnasium[11]API 将平台与 Stable-baselines3[10]集成，用于训练 GNC 任务的强化学习智能体。

因此，本研究的主要贡献在于USV模拟器本身，该模拟器除具备传统模拟器的功能外，还具有以下新颖和/或改进特性：

- 基于巴西真实大型水生生态系统（瓜纳巴拉湾）气象数据推算的风速变异性；
- 与风向及风速耦合的波浪形态；
- 该帆船机器人数字孪生模型通过模拟其在不同环境条件（如风力、波浪及水流）下的动态响应行为，并结合执行器指令，真实反映了实际帆船机器人的运行特性。
- 支持通过Gymnasium和Stable-baselines 3 API进行即用型强化学习（RL）训练的Python库。
- 一个包含所有Yara代码、物理模型、场景、障碍物以及帆船机器人数字孪生体（https://github.com/medialab-fboat/Yara_OVE.git）的GitHub仓库

本文结构安排如下：第2节综述了海洋机器人领域最先进的自主航行系统与虚拟环境；第3节阐述了自主船舶研究中使用部分模拟器的理论原理与工作原理；第4节提出我们的虚拟航行环境方案，并详尽介绍其各项技术细节；第5节通过若干机动测试案例分析虚拟航行机器人的响应特性，并展示三种控制方案在七航点任务中的表现效果；最后在第6节总结研究成果并展望本项目未来研究方向。

## 2 相关作品

本节对当前最前沿的研究成果进行了细致综述，内容涵盖自主航行系统（第2.1节）与海洋机器人虚拟环境（第2.2节）两大领域。

### 2.1 自主航行

本节并非旨在对帆船自主 GNC 进行深入评述与探讨。然而，该主题与我们拟开发的仿真工具具有相关性。

正如Plumet等人所讨论的，为无人帆船构建包含制导、导航与控制系统（GNC）的自主系统，是实现该系统开发过程中的重大挑战。[12]Plumet等[6]Shen等[13]Silva Junior等[14]以及Shen等人的研究[15]与其他任务类似，操控帆船机器人需要执行比机动船更为复杂的动作。作用于帆面的空气动力是其主要推进源，若对这些动力的管理不当，将导致机器人性能下降并引发安全隐患。

risks such as sinking or causing damage to other vessels or people.

In general, the primary approach to mobile robot navigation uses path planning algorithms, and for autonomous sailing robots, it is no different. In 2012, Pêtrés et al. [16] proposed a path-planning solution for unmanned sailing boats based on the artificial potential field algorithm. The authors split the solution into a global and a local potential field to consider the small-scale wind variation in a small region around the boat. Until today, the work of Pêtrés et al. is a reference for the field, being cited several times.

Silva-Junior et al. [14] propose a solution using Q-learning. Although the authors presented good results, their evaluation scenarios are overly simplistic and do not reflect realistic wind behavior or the maneuverability of a sailing boat. Still, the overall results point out that Q-Learning is a promising technique for sailboat path planning.

Lin Zhou et al. [17] present an advanced optimization method, the Beetle Swarm Optimization (IBSO), for autonomous sailboat trajectory planning. The proposed method uses mathematical simulation modeling to validate the IBSO method against Particle Swarm Optimization (PSO) and BSO algorithms regarding convergence speed and accuracy. The focus is on trajectory optimization to avoid obstacles with improved efficiency and precision. The optimized path is defined based on the wind, obstacle distribution, and the goal. The authors presented results using a four-degree-of-freedom (4-DOF) simulator and an actual robot sailing in a controlled environment.

Liu et al. [18] proposes a path-finding algorithm using Q-Learning based on a Gaussian process (GP-QL). The authors sought to improve the adaptability and reliability by modeling the Q-Learning value functions as a Gaussian process. The authors utilized a 3-DOF simulation to conduct their tests. The results show that the method produces a consistent path but exhibits a slightly irregular trajectory.

Shen et al. [15] propose a path-planning solution based on the fusion of Enhanced Adaptive Ant Colony Optimization (ACO) and the Rolling Window Method (RWM) for autonomous sailboats in complex maritime environments. The authors claimed that by combining the global search of ACO with the RWM, the resultant algorithm can provide a dynamic path computation in the presence of moving obstacles. They present results to validate their proposal generated by simulations. Although the results seem to support their claims, the simulation used is simplistic and may not represent a sailing boat's complex motion.

The primary approach to mobile robot navigation typically involves path-planning algorithms. While these solutions have seen significant development, there is still room for improvement. For instance, the QAPF algorithm, proposed by Orozco-Rosas et al. [19], aims to enhance Q-Learning by using artificial potential fields. This approach could potentially be adapted to sailing boats by incorporating the concepts introduced by Pêtrês et al. [16]. Therefore, even though path planning is an important part of an autonomous GNC system, it can effectively respond to small-scale variations in the wave and wind patterns that affect a sailing boat. The complex and strategic operation of the rudder and the sail to enhance performance, avoid collisions, or prevent the boat from capsizing is managed by a control system. This control system can receive sensor data representing the environmental conditions at a given time. Tipsuwan et al. [20] conducts a comprehensive survey on autonomous sailboat control strategies, focusing on dynamic modeling, navigation systems, and control techniques for optimizing course and speed. The work leans more toward theoretical review and categorization of different control approaches used in autonomous sailboats without specifying or presenting a particular solution. Several authors address the sailboat control challenges with different strategies.

Méxas et al. [21] compare reinforcement learning and imitation learning algorithms in autonomous sailboats using the Unity platform and ML-Agents toolkit. The paper explores the performance of Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) under different wind conditions and obstacles, also examining the effect of imitation learning with Behavioral Cloning and Generative Adversarial Imitation Learning. Both algorithms demonstrate good capabilities in controlling the boat under steady wind conditions. The policy generated by PPO presented the most promising results.

Suda and Nikovski [22] describe the application of Deep RL algorithms to determine the optimal decision policy for upwind sailing; wind direction and speed vary according to an unknown stochastic process. The learned policy outperformed baseline control algorithms in simulations considering uncertain and dynamic environments. However, they do not consider the effects of waves, currents, and other environmental factors that can affect a sailboat's performance. They assume that the stochastic process that generates the wind conditions is unknown to the learning algorithm. Still, sailors may have some knowledge or intuition about the wind patterns in a particular area. Additionally, they do not compare the performance of the Deep RL algorithms against other machine learning or control methods used for sailing control.

Liu et al. [23] propose a collaborative control method for the sail and rudder of an unmanned sailboat based on model predictive control. Their model considers the yaw angle to the desired trajectory and the restriction of roll angle as control targets, establishing a four-degree-of-freedom kinematics and dynamics model of the unmanned sailboat. They verified via simulation that the collaborative control has better effects on the yaw angle control and roll angle limitation and can obtain a more accurate path-tracking effect than the traditional separation control method of sail and rudder under

诸如沉没或对其他船舶及人员造成损害等风险。

总体而言，移动机器人导航的主要方法采用路径规划算法，对于自主航行机器人而言亦无例外。2012年，Petres等学者[16]该研究提出了一种基于人工势场算法的无人帆船路径规划方案。研究者将解决方案划分为全局势场与局部势场两部分，以考虑船舶周边小范围内的微尺度风场变化。迄今为止，Petres等人的研究成果仍是该领域的权威参考文献，已被多次引用。

Silva-Junior等[14] 该研究提出了一种基于Q学习的解决方案。尽管作者取得了良好效果，但其评估场景过于简化，未能反映实际风况或帆船的操控特性。不过总体结果表明，Q学习技术在帆船路径规划领域具有广阔应用前景。

林周等[17] 本文提出了一种先进的优化方法——甲虫群优化算法（IBSO），用于自主帆船轨迹规划。该方法通过数学仿真建模，从收敛速度和精度两个维度对 IBSO 算法与粒子群优化算法（PSO）及 BSO 算法进行对比验证。研究重点在于通过轨迹优化实现障碍物规避，显著提升效率与精度。优化路径的确定基于风场分布、障碍物分布及目标位置等因素。研究者采用四自由度（4- DOF）仿真器及实际机器人在受控环境中的航行数据进行了实验验证。

Liu等[18] 提出了一种基于高斯过程（GP-QL）的QL学习路径搜索算法。作者通过将Q学习价值函数建模为高斯过程，旨在提升算法的适应性和可靠性。实验采用3 DOF 仿真进行验证，结果显示该方法能生成稳定路径，但轨迹呈现轻微不规则特征。

Shen等[15] 本文提出了一种基于增强型自适应蚁群优化算法（ACO）与滚动窗口法（RWM）融合的路径规划方案，适用于复杂海事环境中的自主帆船航行。研究者指出，通过将ACO的全局搜索能力与滚动窗口法相结合，所开发算法能够在移动障碍物存在的情况下实现动态路径计算。通过仿真实验验证了该方案的有效性，但需注意所用仿真模型较为简化，可能无法准确反映帆船的复杂运动特性。

移动机器人导航的主要方法通常涉及路径规划算法。尽管这些解决方案已取得显著进展，但仍存在改进空间。例如，Orozco-Rosas等人提出的 QAPF 算法。[19]旨在通过使用人工势场来增强Q学习。该方法可通过整合Petres等人提出的概念，潜在地适用于帆船领域。[16]因此，尽管路径规划是自动驾驶 GNC 系统的重要组成部分，但它能有效应对影响帆船航行的波浪与风况的小尺度变化。舵机与帆的复杂战略操作——包括提升性能、避免碰撞或防止倾覆——均由控制系统管理。该系统可实时接收反映环境状况的传感器数据。Tipsuwan等学者。[20]针对自主帆船控制策略开展了全面调研，重点聚焦动态建模、导航系统及航向与速度优化控制技术。该研究更侧重于理论综述与各类控制方法的分类分析，未提出具体解决方案。多位学者采用不同策略应对帆船控制挑战。

Mexaset al.[21]本研究通过Unity平台与ML-Agents工具包，对比分析了强化学习与模仿学习算法在自主帆船系统中的应用效果。研究重点考察了近端策略优化（PPO）和软体演员-评论家（SAC）算法在不同风况及障碍物环境下的表现，并深入分析了基于行为克隆技术和生成对抗模仿学习的模仿学习方法。实验结果表明，两种算法在稳定风况下的船舶操控能力均表现出色，其中PPO算法生成的策略方案展现出最显著的性能优势。

Suda与Nikovski[22]阐述了深度强化学习算法在确定迎风航行最优决策策略中的应用：风向与风速会根据未知随机过程变化。在考虑不确定动态环境的仿真中，所学策略的表现优于基线控制算法。但该研究未考虑波浪、洋流等可能影响帆船性能的环境因素，且假设生成风况的随机过程对学习算法而言是未知的。尽管如此，水手可能对特定区域的风况模式具备一定认知或直觉判断。此外，该研究未将深度强化学习算法与其他用于航行控制的机器学习方法或控制策略进行性能对比。Liu等人[23] 本文提出了一种基于模型预测控制的无人帆船帆机协同控制方法。该模型以航向角与目标轨迹偏差及横摇角约束为控制目标，建立了无人帆船四自由度运动学与动力学模型。仿真验证表明，相较于传统帆机分离控制方式，协同控制方案在航向角控制与横摇角限制方面效果更优，且能实现更精准的航迹跟踪效果。

the same wind field conditions. Their work does not consider ocean currents' effect on the sailboat's motion.

The works presented in this section propose various approaches to contribute to an autonomous GNC system for sailboats. However, comparing the authors' results is sometimes difficult, as each author used a different kind of simulator. On top of that, the simulators are usually too simplistic and cannot guarantee a reliable representation of a sailboat's motion and maneuverability.

## 2.2 Virtual Environments

In the literature, we could not find a reliable simulation environment for sailing robots that provided a good graphical rendering of the surrounding environment with a high degree of visual realism and all the necessary physics. Several previous works rely on simplified physics models to simulate an environment to support the development and testing of autonomous sailing systems [6, 13, 15, 23, 24]. We noticed that these works had proposed neither a realistic graphical rendering of the surrounding environment nor an accurate simulation of collisions with the inclusion of essential elements such as ocean waves and wind. Some works [5, 25] seek to build a full 3D simulation combining graphical representation and physics. The Kelpie environment proposed by Mendonça et al. [5] seems currently unavailable. The environment proposed by Paravisi et al. [25] combines many wanted elements. Still, its graphical representation is outdated, and the physical model for the boat dynamics and the foil dynamics is not well-suited for a sailing boat.

In the broader context of marine robotics, autonomous boats are rapidly gaining prominence, offering cutting-edge solutions for various applications, from environmental monitoring to maritime logistics. The sophisticated nature of these vehicles requires extensive testing and validation of their control systems, navigation algorithms, and ability to adapt to dynamic marine environments. In this context, virtual simulation environments become fundamental, providing a safe and cost-effective platform for modeling complex scenarios and testing autonomous systems under diverse conditions. Many simulation tools, such as UWSim, VRX, USVSim, and UUV Simulator, propose solutions for autonomous surface vessel design and simulation, playing a significant role in this domain. Each offers distinct features and specialized functionalities, catering specifically to the nuanced requirements of marine robotics and the intricate development process of autonomous boats. Our proposed virtual environment architecture and solution contribute to this evolving landscape, offering an enhanced and comprehensive platform for the development and testing of these sophisticated marine vehicles.

The Underwater Simulator (UWSim) [26] is a solution specifically designed for underwater robotics. In this work, the authors propose a 3D simulator that provides a realistic environment for testing and developing algorithms for underwater vehicles. This includes simulating the dynamics and physics of underwater environments, sensor feedback like sonar and camera, and the interaction of robotic vehicles with water currents and obstacles. For autonomous boats, UWSim can be particularly useful for developing sub-surface components and algorithms, especially if the vessel has functionalities that involve underwater exploration or interaction.

The UUV Simulator [27] is a Gazebo-based simulator focused on Unmanned Underwater Vehicles. The proposal includes a package implementing ROS capabilities for UUV applications. While it's more focused on underwater vehicles, the physics and environmental models may still be relevant for autonomous boats, especially for understanding and simulating the underwater portion of the vehicle or capabilities that cross the boundary between surface and subsurface operations.

Bingham et al. [28] introduce the Virtual RobotX (VRX) simulation, a tool for developing and testing unmanned surface vessels in oceanic environments. The simulated environment focuses on elements used in the RobotX competition tasks and offers sufficient fidelity to enable developers to prototype new solutions. The main boat model available is a differential boat designed to be a digital twin of the real RobotX autonomous vessel. The simulator design adheres to the premise that autonomous behavior that does not work in the simulated world will not work in the physical world, and autonomy that does work in the physical world will work in the simulated world. However, while the simulations provide a high level of fidelity, they may not perfectly replicate real-world conditions, and developers should still test their solutions in physical environments before deployment.

USVSim [25] implements a river-simulated environment with similar characteristics to VRX's. The simulation includes models for boat dynamics, surface water conditions, and interactions between the boat and the water surface. The 6-DOF dynamic model was implemented through the FreeFloat plugin [29]. The authors modify the original FreeFloat to include wave motion from the UWSim, so the boat responds to changes in the water surface. They also modeled the boat's response to environmental disturbances like winds and water currents. The USVSim offers four boat types: rudder boat, airboat, differential boat, and sailboat. All the boats are built from the same hull, and the authors used the Gazebo lift/drag plugin to implement the physics for the rudder, the keel, and the sail. In this way, the sail was modeled as a fixed wing, which does not reflect most of the sailing robots in the literature [23, 30–32].

These four virtual environments implement the vessel dynamic simulations using a 6-DOF model. The UUV and the UWSim focus on underwater vessels, while USVSim and VRX are dedicated to surface vessels. They all enable simu-

相同的风场条件。他们的研究未考虑洋流对帆船运动的影响。

本节展示的研究成果提出了多种方法，旨在为帆船自主 GNC 系统提供支持。然而，由于各研究者采用的模拟器类型不同，结果对比存在困难。此外，现有模拟器通常过于简化，难以准确反映帆船的运动特性与操控性能。

## 2.2 虚拟环境

在现有文献中，我们未能找到能够为航行机器人提供可靠模拟环境的方案——这类环境需具备高视觉真实度的环境图形渲染效果，并完整包含所有必要的物理模拟要素。既往多项研究采用简化物理模型来模拟环境，以支持自主航行系统开发与测试[6, 13, 15, 23, 24]。我们发现这些研究既未实现环境的真实感图形渲染，也未能通过整合海浪、风力等关键要素来精确模拟碰撞行为。部分研究[5, 25]致力于构建融合图形呈现与物理模拟的完整三维仿真系统。Mendonca等人提出的Kelpie环境正是此类研究的典范。[5] 目前似乎不可用。Paravisi等人提出的环境模型[25] 整合了诸多理想要素。然而，其图形化呈现方式已显陈旧，且针对帆船动力学与翼型动力学的物理模型并不适用于实际帆船设计。

在海洋机器人技术的广阔领域中，自主航行器正迅速崭露头角，为环境监测、海上物流等各类应用场景提供尖端解决方案。这类精密设备需要对其控制系统、导航算法以及动态海洋环境适应能力进行全面测试与验证。在此背景下，虚拟仿真环境成为关键支撑，为复杂场景建模及多样化条件下的自主系统测试提供了安全且经济高效的平台。UWSim、 VRX 、USVSim和 UUV Simulator等仿真工具为水面自主航行器设计与模拟提供了解决方案，在该领域发挥着重要作用。各工具均具备独特功能与专业特性，精准满足海洋机器人技术的精细化需求及自主航行器复杂研发流程。我们提出的虚拟环境架构与解决方案为这一发展领域注入新活力，为这类精密海洋设备的研发测试搭建了更完善、更全面的平台。

水下模拟器（UWSim）[26]是专为水下机器人技术量身打造的解决方案。本研究提出了一款三维模拟系统，可为水下载具算法测试与开发提供真实环境。该系统能精准模拟水下环境的动力学特性与物理规律，整合声呐、摄像头等传感器反馈数据，并模拟机器人载具与水流、障碍物的交互作用。对于自主航行艇而言，UWSim在开发水下组件及算法方面具有显著优势，尤其适用于需要执行水下探测或交互任务的船舶场景。

UUV 模拟器[27]是一款基于Gazebo框架的无人水下航行器模拟系统。该方案包含一套实现ROS功能的UUV 应用软件包。尽管其主要针对水下航行器设计，但其中的物理模型与环境模拟技术对自主船舶仍具参考价值，尤其有助于理解与模拟船舶的水下作业环节，以及跨越水面与水下作业边界的技术能力。

Bingham 等 [28] 我们重点介绍 Virtual RobotX（VRX）仿真系统，这是一款专为海洋环境设计的无人水面舰艇研发与测试工具。该仿真环境严格复现RobotX竞赛任务中的核心要素，具备高度真实感，可助力开发者快速搭建创新解决方案原型。系统标配的差动式船舶模型，正是为真实RobotX自主舰艇打造的数字化孪生体。仿真器设计遵循“仿真环境失效即物理环境失效”的核心原则——在虚拟环境中表现优异的自主系统，实际应用中未必可靠；反之，物理环境中表现稳定的系统，在仿真环境中同样可能失效。尽管仿真技术能提供高度逼真的模拟体验，但其仍存在无法完全复现真实环境条件的局限性，开发者在实际部署前仍需通过物理环境进行验证测试。

USVSim[25]构建了一个与 VRX 研究中相似特性的河流模拟环境。该模拟系统包含船舶动力学模型、水面水体条件模型以及船舶与水面相互作用模型。通过FreeFloat插件[29]实现了6 DOF 动力学模型。研究者对原始FreeFloat软件进行修改，整合了UWSim平台的波浪运动数据，使船舶能够响应水面状态变化。同时建立了船舶对风力、水流等环境扰动的响应模型。USV-Sim提供四种船舶类型：舵船、气垫船、差速船和帆船。所有船舶均采用统一船体结构，研究者通过Gazebo升力/阻力插件实现了舵机、龙骨及帆的物理模拟。其中帆部采用固定翼模型构建，这与现有文献中大多数航行机器人设计存在差异[23, 30–32]。

这四个虚拟环境均采用六自由度（6- DOF）模型实现船舶动力学仿真。其中 UUV 和UWSim专注于水下船舶仿真，USV-Sim与 VRX 则专用于水面船舶仿真。所有仿真平台均支持...

lated sensors and interface with ROS. However, the only one that offers a sailing boat model and parameters is USVSim.

Surface vessel simulations are not new, but an open-source, reliable sailing vessel simulation is hard to find. Despite its many qualities, the sailing vessel simulation offered within USVSim has poorly modeled aerodynamic forces, boat dynamics, and sail behavior. The only contribution of the aerodynamic force seems to be the forward component, and the sail is modeled as an airplane wing. We do not observe the torque or lateral drifty caused by the lateral component of the aerodynamic force. In an actual sailing boat, the effects caused by the lateral component are resisted by the keel, but the boat still suffers some of its influence. Building on that, our work introduces an innovative virtual environment architecture and solution designed as an open platform for reinforcement learning tasks for USVs, focusing on sailing robots. We build a sailing robot digital twin, implementing reliable physics to simulate the boat dynamics and response to aerodynamic, hydrodynamic, and hydrostatic forces. Our solution provides full access to most physics simulation variables, computer vision tasks, and a simple and direct way to use RL APIs like Gymnasium [11] and Satble-baelines3 [10].

## 3 Physics Simulation for Sailing Boats

Most simulation engines have solutions for fast simulation of rigid body physics and force field computation. To simulate a surface vessel, one must add hydrodynamic, hydrostatic, and aerodynamic forces to the rigid body physics model. This Section discusses the required physics to simulate a sailing robot in different wind and sea surface conditions. A reliable physics simulation is required to improve the chances a GNC solution developed in a virtual world could work in the real world [28].

A sailboat sails under the influence of various hydrodynamic and aerodynamic forces. Its main propulsion is produced by the force of the wind on the sail. This force is generated by the sail trim (sail camber), which shapes the sail like an airfoil. The curvature of the sail causes a change in the airflow direction, which causes a reaction force called lift. A similar force also appears on the rudder and the keel when the boat moves through the water. Thus, to implement the digital twin (in the simulator), it is necessary to compute different forces acting on different parts of the boat.

Theoretical aspects of the physics simulation include aerodynamic and hydrostatic forces, current, and wind, among other physics constraints that act on the whole system. These aspects are further described in the next subsections so the reader understands the key aspects of modeling the physics of a simulated sailing robot.

### 3.1 Boat Dynamic Model

The motion of a boat traveling through the water can be described with the help of a six-degrees-of-freedom (6-DOF) vectorial model for marine crafts [33]. The motion equation has the following form:

$$\underbrace{\mathbf{M}_A\dot{\boldsymbol{\nu}}_r + \mathbf{C}_A(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r + \mathbf{D}(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r}_{\text{hydrodynamic forces}} + \underbrace{\mathbf{g}(\boldsymbol{\eta})}_{\text{hydrostatic forces}} +$$

$$\underbrace{\mathbf{M}_{RB}\dot{\boldsymbol{\nu}} + \mathbf{C}_{RB}(\boldsymbol{\nu})\boldsymbol{\nu}}_{\text{rigid body forces}} = \tau_{propeller} + \tau_{wind} + \tau_{waves} \quad (1)$$

where

$$\boldsymbol{\eta} = [x, y, z, \phi, \theta, \psi]^T \quad (2)$$

$$\boldsymbol{\nu} = [u, v, w, p, q, r]^T \quad (3)$$

The position ($\boldsymbol{\eta}$) and velocity ($\boldsymbol{\nu}$) vectors include the USV 6 degrees-of-freedom: surge $u$, sway $v$, heave $w$, roll $p$, pitch $q$, and yaw $r$ (Fig. 1). While the velocities $\boldsymbol{\nu}$ are described in the robot's frame $b$, the robot position $\boldsymbol{\eta}$ is related to a reference frame that could be Earth-centred, Earth-fixed, fixed on the water surface, fixed on a base station, etc. Table 1 depicts the notation for position an velocities [34]

The hydrodynamic forces in Eq. 1 describe forces generated by the water flowing around the hull as the boat moves. These forces include an inertial and a drag term. When the boat moves, the contact between the hull and the water causes resistance due to radiation-induced potential damping, skin friction, lifting forces, vortex shedding, and wave drifting [33]. The effects of the resistance forces are compiled in the hydrodynamic damping matrix $\boldsymbol{D}(\boldsymbol{\nu}_r)$.

Friction between the hull and the water causes water shear, i.e., part of the water near the contact surface will gain velocity. In other words, as the boat moves, it "pulls" a certain mass of water. The influence of this water movement on the boat

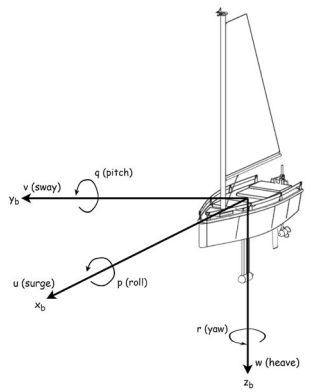



**Fig. 1** The 6-DOF velocities $u$, $v$, $w$, $p$, $q$ and $r$ in the body-fixed reference frame $b = (x_b, y_b, z_b)$

配备传感器并支持与ROS接口。然而，唯一提供帆船模型及参数的工具是USVSim。

水面舰艇仿真技术并非新兴领域，但开源且可靠的帆船仿真软件却凤毛麟角。尽管USVSim提供的帆船仿真系统具备诸多优势，其对空气动力学作用力、船舶动力学特性及帆具行为的建模仍存在明显不足。该仿真系统仅能准确模拟空气动力学作用力的前向分量，且将帆具建模为飞机机翼结构，未能有效捕捉由空气动力学侧向分量引发的扭矩效应及横向漂移现象。在实际帆船航行中，虽然龙骨结构能有效抵消侧向分量的影响，但船舶仍会受到部分作用力干扰。基于此，本研究创新性地构建了面向水面舰艇强化学习任务的开放式虚拟环境架构与解决方案，重点针对航行机器人展开开发。我们通过建立航行机器人数字孪生模型，采用可靠的物理仿真技术，精准模拟船舶动力学特性及其对空气动力、流体动力和静水压力的响应行为。我们的解决方案可全面访问大多数物理模拟变量和计算机视觉任务，并提供一种简单直接的方法来使用Gymnasium[11]和Satblebaelines3[10]等强化学习API。

## 3 帆船运动的物理模拟研究

大多数仿真引擎都具备快速模拟刚体物理过程和力场计算的功能。要对水面舰艇进行仿真，必须在刚体物理模型中加入流体动力学、静力学及空气动力学作用力。本节重点探讨在不同风况与海面条件下模拟航行机器人所需的物理模型。可靠的物理仿真技术对于提升虚拟世界中开发的 GNC 解决方案在现实环境中的适用性至关重要[28]。

帆船在航行过程中会受到多种流体动力学和空气动力学作用力的影响。其主要推进力来源于风对帆布产生的作用力，这种作用力通过帆的调整装置（即帆的曲率设计）形成，该装置将帆面塑造成类似翼型的形状。帆面的曲率会导致气流方向发生改变，从而产生被称为升力的反作用力。当船只在水中航行时，舵面和龙骨也会产生类似作用力。因此，在模拟器中实现数字孪生技术时，必须对作用于船体不同部位的各种作用力进行精确计算。

物理模拟的理论层面涵盖气动力、静水力、水流及风力等作用于整个系统的物理约束条件。下文将通过各子章节对这些要素进行详细阐述，帮助读者掌握模拟航行机器人物理建模的核心要点。

### 3.1 船舶动力学模型

船舶在水中航行时的运动可通过六自由度（6- DOF）矢量模型进行描述[33]。其运动方程具有以下形式：

$$\underbrace{\mathbf{M}_A\dot{\boldsymbol{\nu}}_r + \mathbf{C}_A(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r + \mathbf{D}(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r}_{\text{水动力}} + \underbrace{\mathbf{g}(\boldsymbol{\eta})}_{\text{静水压力}} + \cdots$$

$$\underbrace{\mathbf{M}_{RBD}\dot{} + \mathbf{CRB}(\boldsymbol{D})\boldsymbol{D} - = \boldsymbol{\tau}_{螺旋桨} + \boldsymbol{\tau}_{风} + \boldsymbol{\tau}_{波浪}}_{\text{刚体力}} \quad (1)$$

在哪里

$$\boldsymbol{\eta} = [x, y, z, \phi, \theta, \psi]^T \quad (2)$$

$$\boldsymbol{\nu} = [u, v, w, p, q, r]^T \quad (3)$$

位置向量（$\boldsymbol{\eta}$）和速度向量（$\boldsymbol{D}$）包含USV的6个自由度：纵摇$u$、横摇$v$、垂向运动$w$、横滚$p$、俯仰$q$以及偏航$r$（图1）。虽然速度$\boldsymbol{D}$以机器人坐标系$b$描述，但机器人位置$\boldsymbol{\eta}$需关联参考坐标系——该坐标系可为地心坐标系、地球固定坐标系、水面固定坐标系或基站固定坐标系等。表1展示了位置与速度的符号表示法[34]。

等式1中的流体力学力描述了船只移动时水体绕船体流动所产生的力。这些力包括惯性力和阻力项。当船只移动时，船体与水体之间的接触会因辐射诱导的潜在阻尼、表面摩擦、升力、涡旋脱落和波浪漂移而产生阻力[33]。阻力力的影响被汇总在流体动力阻尼矩阵$\boldsymbol{D}(\boldsymbol{Dr})$中。

船体与水面之间的摩擦会产生水剪切力，即接触表面附近的部分水体会获得速度。换言之，当船舶运动时，会带动一定质量的水体。这种水体运动对船舶的影响

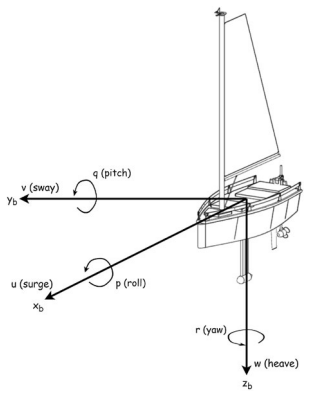



**图1**体固定参考系$b$=(xb，yb，zb)中的6- DOF 速度$u$、$v$、$w$、$p$、$q$和$r$

**Table 1** The SNAME's notation for marine vessels [34]

| DOF | | Linear and angular velocities | Positions and Euler angles |
|---|---|---|---|
| 1 | motion in the $x$ direction ($surge$) | $u$ | $x$ |
| 2 | motion in the $y$ direction ($sway$) | $v$ | $y$ |
| 3 | motion in the $z$ direction ($heave$) | $w$ | $z$ |
| 4 | rotation about the $x$ axis ($roll$, heel) | $p$ | $\phi$ |
| 5 | rotation about the $y$ axis ($pitch$, trim) | $q$ | $\theta$ |
| 6 | rotation about the $z$ axis ($yaw$) | $r$ | $\psi$ |

Adapted from Fossen's book [33]

motion is represented as an additional mass. In Eq. 1, the additional mass and its inertial effect are described by the added mass matrix $\mathbf{M}_A$ and the Coriolis-centripetal matrix $\mathbf{C}_A(\boldsymbol{\nu}_r)$ [28, 33].

The hydrostatic force acting on the hull can be simplified as the buoyancy force caused by the difference in pressure between the water line and the deepest part of the hull. Archimedes' principle says that the buoyancy force acting on a body equals the weight of water displaced by the body when totally or partially submerged (Fig. 2).

In a flat water surface, the water line says how much of our boat should be submerged so the buoyancy force matches the boat's weight making it float in equilibrium. However, in the presence of waves, the water surface is moving, and consequently, the water pressure distribution on the hull changes over time (Fig. 3).

## 3.2 Aerodynamic Forces

Under the presence of wind, the main sail of a sailing boat acts as an airfoil, changing the airflow direction and generating two aerodynamic forces: the lift ($L$) and the drag ($D$). These two forces depend on the respective lift and drag coefficients ($C_L$ and $C_D$), the velocity of the air flowing by the sail ($V_A$), the sail area ($A$), the angle between the sail and the airflow ($\alpha$ - attack angle), and the air density ($\rho_{air}$). Equations 4 and 5

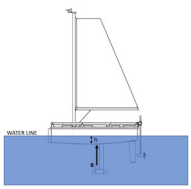



**Fig. 2** Buoyancy on a flat water surface

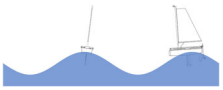


**Fig. 3** Wave effect on the boat equilibrium

depict the modulus of the resultant lift and drag forces acting on the sail. The lift and drag coefficients are intrinsic to the sail shape and material and are functions of the attack angle ($\alpha$). Figure 4 shows the aerodynamic forces as a function of the attack angle.

$$L = \frac{1}{2}\,\rho_{air}\,V_A^2\,A\,C_L(\alpha) \tag{4}$$

$$D = \frac{1}{2}\,\rho_{air}\,V_A^2\,A\,C_D(\alpha) \tag{5}$$

As a sailing boat moves, an airflow is generated through the sail that depends on the boat velocity ($V_B$) and the wind speed in the resting referential ($V_T$). The speed of the resultant airflow is called apparent wind speed ($V_A$). Figure 5 depicts a simple vector representation of the true and apparent wind speeds and the lift and drag forces. The resultant aerodynamic forces on the sail ($R$) can be decomposed into a forward ($F_{fwr}$) component that propels the boat forward (surge) and a lateral component ($F_{LAT}$) that pushes the boat sideways (sway).

## 3.3 Hydrodynamic Forces on Under Water Elements

To complete the whole dynamic model of a sailing boat, a key aspect is the hydrodynamic forces on the keel and the rudder. The keel's main objective is to balance the effects caused by the resultant lateral force acting on the main sail. The boat suffers from two main effects: a sideways movement and a rotation (Fig. 6). By design, the keel has a weight in its bottom part and a symmetric foil shape. While the extra

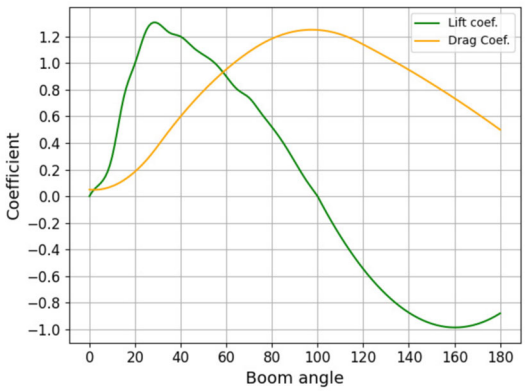



**Fig. 4** Lift and Drag forces as a function of the attack angle

**表1**海上船舶的 SNAME 符号[34]

| DOF | | 线性与角速度 | 位置和欧拉角 |
|---|---|---|---|
| 1 | x方向运动（*冲浪*） | $u$ | $x$ |
| 2 | y方向的运动（*摆动*） | $v$ | $y$ |
| 3 | z方向运动（*起升*） | $w$ | $z$ |
| 4 | 绕x轴旋转（*侧翻*，脚跟） | $p$ | $\phi$ |
| 5 | 绕y轴旋转（*俯仰*，调整） | $q$ | $\theta$ |
| 6 | 绕z轴旋转（*偏航*） | $r$ | $\psi$ |

改编自Fossen的著作[33]

运动被表示为附加质量。在等式1中，附加质量及其惯性效应通过附加质量矩阵**MA**和科里奥利向心矩阵**CA**($\boldsymbol{v}_r$)[28, 33]进行描述。

作用在船体上的静水压力可简化为水线与船体最深处之间压力差所产生的浮力。阿基米德原理指出，作用在物体上的浮力等于该物体完全或部分浸没时所排开水的重量（图2）。

在平静的水面上，水线应使我们船只的大部分浸没，从而使浮力与船体重量相等，保持平衡漂浮状态。然而当存在波浪时，水面会随波运动，导致船体表面的水压分布随时间发生变化（图3）。

## 3.2 空气动力学作用力

在风力作用下，帆船的主帆会像翼型一样改变气流方向，产生两种空气动力：升力（*L*）和阻力（*D*）。这两种力取决于相应的升力系数和阻力系数（*CL*与*CD*）、流经帆面的空气速度（*VA*）、帆面积（*A*）、帆面与气流之间的夹角（$\boldsymbol{\alpha}$——迎角）以及空气密度（$\boldsymbol{\rho}$）。公式4和5描述了作用于帆面上的合成升力与阻力的模量。升力系数和阻力系数与帆面形状及材料特性密切相关，并随迎角（$\boldsymbol{\alpha}$）变化。图4展示了空气动力随迎角变化的函数关系。

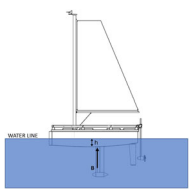



**图2**平水面浮力

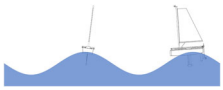


**图3**波浪对船舶平衡状态的影响

$$L=\rho_{air}\frac{1}{2}V_A^2 A\, C_L(\alpha) \tag{4}$$

$$D=\rho_{air}\frac{1}{2}V_A^2 A\, C_D(\alpha) \tag{5}$$

当帆船运动时，帆面上会产生气流，该气流取决于船速（*VB*）和静止参考系中的风速（*VT*）。合成气流的速度称为表观风速（*VA*）。图5展示了真实风速与表观风速以及升力和阻力的简单矢量表示。作用在帆上的合成空气动力（*R*）可分解为推动船体前进的前向分量（*Ffwr*）（涌浪）和使船体横向移动的侧向分量（*FLAT*）（横摇）。

## 3.3 水下元件上的流体动力

要构建帆船的完整动力学模型，关键在于研究龙骨与舵面所受的流体动力作用。龙骨的核心功能在于平衡主帆所受横向合力产生的影响。帆船主要面临两个关键问题：横向位移与旋转运动（见图6）。根据设计原理，龙骨底部配置有配重块并采用对称翼型结构。当额外

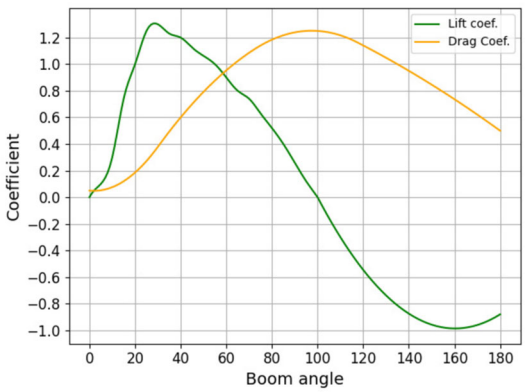



**图4**吊力与阻力随攻角变化关系

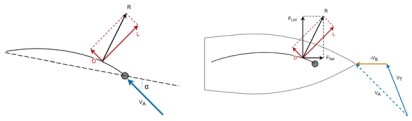


**Fig. 5** Wind forces acting on the main sail ($L$ and $D$) and being transmitted to the boat ($F_{fwr}$ propelling the boat forward, and $F_{LAT}$ pushing the boat sideways)

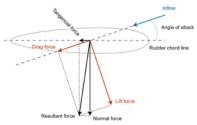


**Fig. 7** Force decomposition in the rudder

weight helps to lower the center of gravity of the sailing robot, it also opposes the rotational motion caused by the lateral component of the aerodynamic forces. On the other hand, its foil shape produces lift and drag forces as the boat moves through the water and resists the sideways motion caused by the aerodynamic forces on the sail. In Fig. 6, the water hydrodynamic resistance force $\mathbf{R}_{LAT}$ contains contributions from the resistance of the water to the lateral movement of the hull, the keel, and the rudder.

The rudder is designed as a symmetric foil to produce lift and drag forces as the boat moves. The resultant of these induced forces generates a torque in the hull that modifies its direction. As we turn the rudder, we modify the angle of attack, i.e., the angle between the fluid velocity and the rudder chord line. By changing the attack angle, we modify the resultant force, increasing or decreasing the torque transmitted to the boat hull. Figure 7 presents a simple illustration of the force decomposition in the rudder. The inflow or fluid (water) velocity in Fig. 7 is mainly caused by the boat's motion; in the boat's referential, the water moves with a velocity equal to the boat's velocity but in the opposite way. However, natural water velocity can occur when the boat travels under waves and/or currents. Therefore, as for the wind, the inflow represents an apparent water motion speed composed of the natural water movement and the movement due to the boat motion.

## 4 Proposed Sailing Virtual Environment

Our proposed simulated environment extends the functionalities of the Gazebo Sim [9] to support sailing robots. A key to achieving this goal is representing the influence of both water and air on the robot's behavior. We implement new physics models to describe the forces acting on the sail, the rudder, and the keel. We adapt the wave and dynamic models implemented by the VRX project [28] to reflect the unique properties of a sailing robot. In this way, the movement of our sailing robot digital twin is subjected to the ocean waves and wind conditions, as well as to the aerodynamic forces acting on the sail. Each moving part in the digital twin is treated as a different component with its own center of gravity and mass distribution. In this way, the boat's balance is affected by the waves and the forces acting on its different

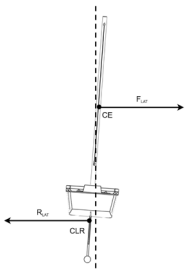



**Fig. 6** Aerodynamic lateral force component $\mathbf{F}_{LAT}$ and the hydrodynamic residence force $\mathbf{R}_{LAT}$



| Item | Description |
|---|---|
| Length | 2500 mm |
| Width | 830 mm |
| Draught | 1322 mm |
| Mast length | 3130 mm |
| Hull weight | 65 kg |
| Keel weight | 37 Kg |
| Batteries weight | 25.50 Kg + 25.50 Kg |
| Number compartments | 4 |
| Number waterproof wallets | 7 |
| Sail area | 3188320.84 $mm^2$ |
| Sail type | Sailcloth - Dacron |
| Stored power | 105 A + 105 A (12V) |
| Solar panel power | 100 Wp |
| Rearch group | UFF/RJ and UFRN/RN |

**Fig. 8** The F-Boat (top left), the E-Boat (top right), and specification table (bottom)

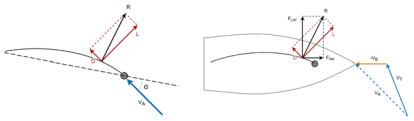


**图5**作用于主帆（*L*和*D*）并传递至船体的风力（*Ff wr*使船体向前推进，*FLAT*使船体横向偏移）

重量有助于降低帆船机器人的重心，同时还能抵消由空气动力横向分量引起的旋转运动。另一方面，其翼型结构在船只航行时产生升力和阻力，有效抵抗风帆受空气动力作用产生的横向运动。如图6所示，水动力阻力**RLAT**包含三个主要分量：水体对船体横向运动的阻力、龙骨阻力以及舵面阻力。

舵的设计采用对称翼型结构，可在船舶运动时产生升力与阻力。这些诱导力的合力会在船体中产生扭矩，从而改变其运动方向。通过转动舵面，我们可调节攻角（即流体速度与舵弦线之间的夹角）。通过调整攻角，能够改变合力大小，进而调节传递至船体的扭矩。图7展示了舵部受力分解的简明示意图。图7中所示的来流（水流）速度主要源于船舶运动：在船舶参考系中，水流以与船速相同但方向相反的速度运动。然而当船舶在波浪或水流中航行时，实际水体速度可能发生变化。因此与风力作用类似，来流速度本质上是由自然水流运动与船舶运动共同构成的表观水体运动速度。

## 4 拟建航海虚拟环境

我们提出的模拟环境扩展了Gazebo Sim[9]的功能，以支持航行机器人的研究。实现这一目标的关键在于准

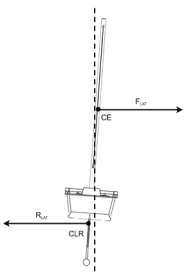



**图6**气动侧向力分量**FLAT**与流体静止阻力**RLAT**

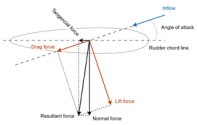
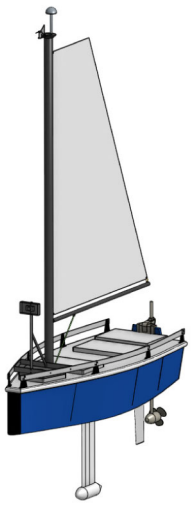


**图7**方向舵中的力分解

确表征水体与空气对机器人行为的影响。我们通过引入新型物理模型来描述作用于帆面、舵面及龙骨的力场，并对 VRX 项目[28]中实现的波浪与动力学模型进行适配，以体现航行机器人特有的物理特性。由此，我们的航行机器人数字孪生体运动不仅受海洋波浪与风力条件影响，还需应对作用于帆面的空气动力学作用力。数字孪生体中的每个运动部件均被视作具有独立重心与质量分布的独立组件，从而确保船舶平衡状态能实时响应波浪冲击及各作用力的共同影响。





| 物品 | 描述 |
|---|---|
| 长度 | 2500毫米 |
| 宽度 | 830毫米 |
| 干呕 | 1322毫米 |
| 桅杆长度 | 3130毫米 |
| 船体重量 | 65公斤 |
| 龙骨重量 | 37公斤 |
| 电池重量 | 25.50 Kg + 25.50 Kg |
| 隔间数量 | 4 |
| 防水钱包数量 | 7 |
| 帆面积 | 3188320. 84毫米 $^2$ |
| 帆型 | 帆布 - 聚酯纤维 |
| 储存功率 | 105 A + 105 A (12V) |
| 太阳能电池板功率 | 100瓦峰值功率 |
| 研究组 | UFF /RJ与 UFRN /RN |

**图8** F型艇（左上）、E型艇（右上）及规格表（底部）

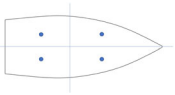


**Fig. 9** Points where the buoyancy force is applied

components. Through this Section, we describe our proposed virtual environment and all its features.

## 4.1 Sailing Boat Model

Our simulated sailing robot model, named E-Boat, was built to be the digital twin of our real sailing robot named F-Boat [3]. As such, E-Boat shares the same physical characteristics, actuators, and F-Boat's sensors (see Fig. 8).

In the simulation, the E-Boat's main source of propulsion is the wind, but it also relies on an electric propeller when needed. To describe its motion through the water, we must consider the action of rigid body static and dynamic forces as much as contributions from hydrodynamic, hydrostatic, and aerodynamic forces (see Section 3). We approximate the simulation of this motion using an adaptation of the Fossen six-degrees-of-freedom (6-DOF) vectorial model for marine crafts (Eq. 1) implemented and maintained by the VRX project team [28].

For our application, the wave motion of the water surface influences the motion control and the perception of the sensors, like images captured by cameras. For this reason, we kept a simplified implementation where the boat's response to the wave motion is restricted to heave, roll, and pitch velocities. The algorithm provided by Bingham et al. [28] emulates the unbalance caused by a wave by splitting the hull in a grid with size $2 \times N$, where the user can select $N$. The buoyancy force is computed for each boat segment, enabling responses as illustrated in Fig. 3. For the E-Boat, we use $N = 2$ (Fig. 9).

Although the 6-DOF model from VRX is suitable for describing the hull behavior, it is missing four vital hydrodynamic forces for a sailing boat: the lift and the drag generated by the water moving around the keel and the rudder (see Section 3.3). Thus, we modify the original VRX model by adding these four extra forces.

As the main source of propulsion is the wind, we also added the aerodynamic force acting on the sail. In the original VRX model, the wind force ($\tau_{wind}$) interacts only with the hull. In our model, the wind is commissioned as a main propulsion force, and its contribution is computed as described in Section 3.2.

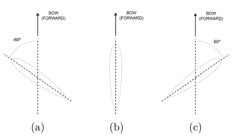


**Fig. 10** Rudder maximum positions (a) and (c), and resting position (b)

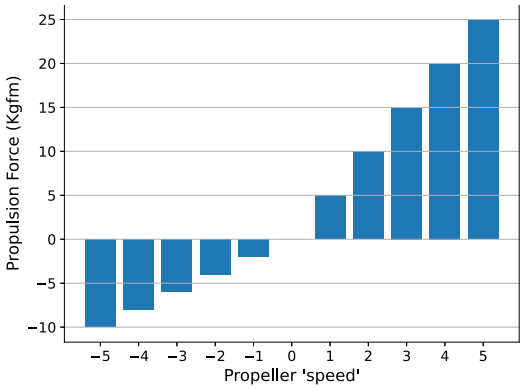



**Fig. 11** Propulsion force intensity as a function of the propeller speed

### 4.1.1 Actuators and Control Interfaces

The E-Boat navigation is performed by controlling three actuators: the boom to which the sail is attached, the rudder, and the electric propeller. Within the Yara OVE the E-Boat's actuators were defined to emulate the behavior of those of the F-Boat.

The rudder can take any position from -60 to +60 degrees (Fig. 10), and a proportional controller is used to set the angle as required. The controller defines the angular velocity of the rudder, and the Gazebo physics engine applies the necessary force.

The propeller control interface implements a discrete function with five forward and five backward speeds. For each of these ten speeds, a force is applied to the center of the propeller joint. The intensity of the applied force is set according to the propeller manufacturer manual [35], and it is depicted in Fig. 11. Figure 12 illustrates the propulsion force generated by the propeller. As the force is applied on the joint connecting the propeller and the electric engine body, it is as if, when activated, the propeller is pushing the engine body.

While the rudder and the propeller are operated by engines acting on their respective joints, the boom operation differs. In the F-Boat (the real sailing robot), the boom is connected to a cable that can be released or pulled using an electric



**Fig. 12** The red arrow indicates the force the propeller generates

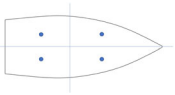


**图9**浮力作用点

组件。通过本节，我们将阐述所提出的虚拟环境及其所有功能特性。

### 4.1 帆船模型

我们构建的模拟航行机器人模型E-Boat，旨在作为真实航行机器人F-Boat的数字孪生体[3]。因此，E-Boat与F-Boat具有相同的物理特性、执行器及传感器配置（参见图8）。

在模拟中，E-Boat的主要推进源是风力，但在需要时也会依赖电动螺旋桨。为了描述其在水中的运动，我们必须同时考虑刚体静力和动力的作用，以及流体动力、静水力和空气动力的贡献（参见第3节）。我们采用 VRX 项目团队实施和维护的Fossen六自由度（6-DOF）船舶矢量模型（1）的改进版本来近似模拟这种运动[28]。

在本应用中，水面波浪运动会影响运动控制及传感器感知（如摄像头采集的图像）。为此，我们采用简化实现方案，将船舶对波浪运动的响应限制为上下运动、横摇运动和纵摇运动速度。该算法由Bingham等人提供。[28]通过将船体划分为尺寸为2× $N$的网格来模拟波浪引起的不平衡，其中用户可选择$N$。针对每个船体段计算浮力，从而实现如图3所示的响应效果。对于E-Boat，我们采用$N$=2（图9）。

尽管 VRX 提出的6 DOF 模型适用于描述船体行为，但该模型遗漏了帆船运动中四个关键水动力：由龙骨和舵周围水流产生的升力与阻力（参见3.3章节）。因此，我们通过添加这四个额外力对原始 VRX 模型进行了改进。

由于主要推进源是风力，我们还加入了作用于帆上的空气动力。在原始 VRX 模型中，风力（$\varrho_{风}$）仅与船体相互作用。在我们的模型中，风被设定为主要推

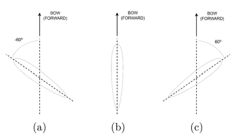



**图10**方向舵最大位置(a)与(c)，以及静止位置(b)

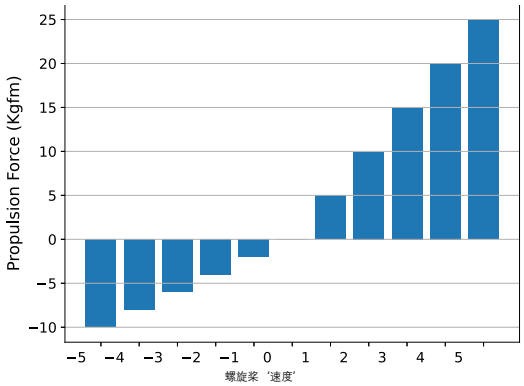



**图11**推进力强度与螺旋桨转速的关系

进力，其贡献度计算方法如第3.2节所述。

#### 4.1.1 执行器与控制接口

E艇的导航系统通过控制三个执行器实现：悬挂帆的横杆、舵机以及电动螺旋桨。在Yara OVE中，E艇的执行器被设计为模拟F艇的执行器行为。

舵机可在-60度至+60度范围内调节角度（图10），并通过比例控制器实现精准角度设定。该控制器可精确控制舵机角速度，而Gazebo物理引擎则负责施加相应作用力。

螺旋桨控制接口实现了具有五档前进速度和五档倒车速度的离散功能。针对这十种速度中的每一种，都会在螺旋桨关节中心施加作用力。所施加力的强度根据螺旋桨制造商手册[35]进行设定，并如图11所示。图12展示了螺旋桨产生的推进力。当力作用于螺旋桨与电动机本体连接的关节时，其效果相当于螺旋桨启动时正在推动发动机本体。虽然舵机与螺旋桨通过作用于各自关节的电机进行驱动，但吊杆的操作方式有所不同。在F型帆船（真实帆船机器人）中，吊杆通过可释放或拉动的电缆与电动机本体相连。



**图12**红色箭头表示螺旋桨产生的力

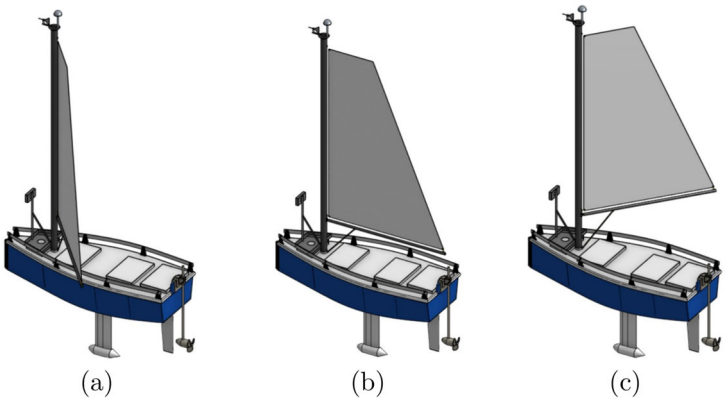



**Fig. 13** Sail opening to left (a), right (c), and closed in its resting position (b)

motor. The motor does not act on the joint that connects the boom to the mast. Under the influence of the wind, the boom moves to the left or the right, and the length of the boom cable controls the amplitude of this movement. Figure 13 illustrates different boom positions. In our simulated environment, the emulation of the actual boom controller is implemented as described in Algorithm 1. At line 1, we set the boom angular velocity, defined by the cable releasing rate. As the algorithm is executed in every simulation loop (the loop in which the Gazebo physics engine computes the next state of the models, see Fig. 18), the value $sov = 0.29$ means that the boom cable releasing/pulling rate is twenty-nine degrees per second. Using the upper and lower limits of the boom joint (lines 6, 7, 9, and 10), we represent the restriction imposed by the cable to the boom motion inside Gazebo. The $required_boom_angle$ must be a positive value between 0 and 90 degrees. Each time a new $required_boom_angle$ is set, the algorithm adjusts the boom joint limits. In this way, the boom is free to move with the wind from right to left, only limited by the absolute value of the $required_boom_angle$ (Fig. 13). Algorithm 1 is implemented within the Boom Controller Plugin, and it is executed in every physics engine time step, as we will detail in Section 4.2.

**Algorithm 1** Emulate boom operation. See text for explanations.

1: $sov \Leftarrow 0.29$
2: **if** $required_boom_angle > 90.0$ **then**
3: $\quad required_boom_angle \Leftarrow 90.0$
4: **end if**
5: **if** $required_boom_angle > joint_upper_limit + sov$ **then**
6: $\quad joint_upper_limit \Leftarrow joint_upper_limit + sov$
7: $\quad joint_lower_limit \Leftarrow joint_lower_limit - sov$
8: **else if** $required_boom_angle < joint_upper_limit - sov$ **then**
9: $\quad joint_upper_limit \Leftarrow joint_upper_limit - sov$
10: $\quad joint_lower_limit \Leftarrow joint_lower_limit + sov$
11: **end if**
12: **if** $joint_upper_limit > 90.0$ **then**
13: $\quad joint_upper_limit \Leftarrow 90.0$
14: $\quad joint_lower_limit \Leftarrow -90.0$
15: **end if**

#### 4.1.2 Sensors

The virtual sensor array within the E-Boat measures apparent wind speed and its direction, the linear velocity in the bow direction (surge, see Fig. 1), and roll angle. The wind sensors are positioned at the top of the mast (Fig. 14), while the other sensors are assembled below the deck. A stereo camera is assembled at the bow to complete our sensor array (Figs. 15 and 16).

### 4.2 System Architecture

The proposed virtual sailing environment is implemented as a ROS package. We use ROS launch files to launch the virtual world in Gazebo as ROS nodes and populate them with models. Currently, our virtual world can be populated

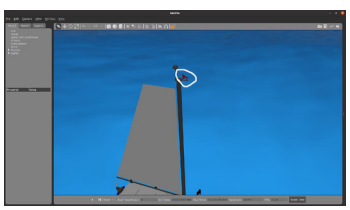


**Fig. 14** Picture of the wind senior on our Virtual World

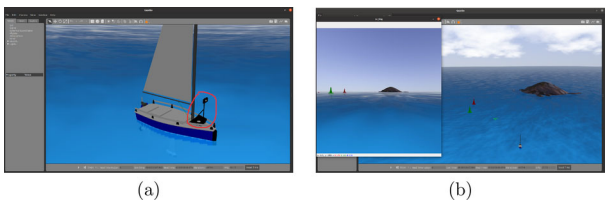



**Fig. 15** Bow camera: (a) location of the camera. (b) Example of a scene provided by the camera

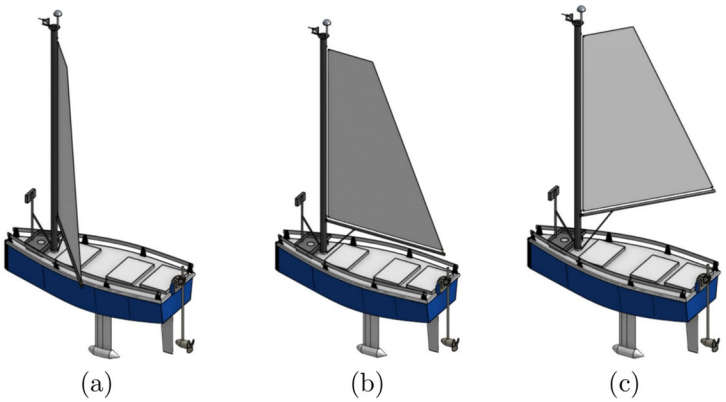



**图13**帆向左开启(a)、向右开启(c)及静止状态下的闭合状态(b)

电机。该电机不作用于连接吊臂与桅杆的关节处。在风力影响下，吊臂会向左或向右移动，吊臂缆绳的长度控制着这种运动的幅度。图13展示了不同吊臂位置。在我们的模拟环境中，实际吊臂控制器的仿真实现方式如算法1所述。在第1行中，我们通过缆绳释放速率定义吊臂角速度。由于算法在每次模拟循环中执行（该循环中Gazebo物理引擎计算模型的下一状态，见图18），数值*sov*=0.29表示吊臂缆绳释放/拉动速率为每秒二十九度。利用吊臂关节的上下限值（第6、7、9和10行），我们在Gazebo中表征了缆绳对吊臂运动施加的约束。*所需_吊臂_角度*必须为0至90度之间的正值。每当设定新的*所需_吊臂_角度*时，算法会调整吊臂关节的限制范围。通过这种方式，吊臂可随风向自由从右向左移动，仅受*所需_吊臂_角度*绝对值的限制（图13）。算法1实现在吊臂控制器插件中，并在每个物理引擎时间步执行，具体细节将在第4.2节详述。

### 4.1.2 传感器

E-Boat船内的虚拟传感器阵列可测量表观风速及其方向、船首方向的线速度（涌浪，见图1）以及横摇角。

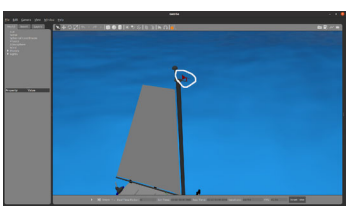


**图14**我们虚拟世界中的风力发电机图片

**算法1**模拟吊臂操作。具体说明参见正文。

```
1: sov ⇐ 0.29
2:如果required_boom_angle >90.0则
3：required_boom_angle ⇐ 90.0
4：end if
5：如果required_boom_angle > joint_upper_limit + sov则
6:    joint_upper_limit ⇐ joint_upper_limit + sov
7：joint_lower_limit ⇐ joint_lower_limit - sov
8：else if required_boom_angle < joint_upper_limit - sov then
9：joint_upper_limit ⇐ joint_upper_limit - sov
10：joint_lower_limit ⇐ joint_lower_limit + sov
11：结束 if
12：如果joint_upper_limit >90.0则
13：joint_upper_limit ⇐ 90.0
14：joint_lower_limit ⇐ -90.0
15：end if
```

风速传感器安装在桅杆顶部（图14），其余传感器则集成于甲板下方。为完善传感器阵列配置，我们在船首安装了立体相机（图15和16）。

## 4.2 系统体系结构

所提出的虚拟航行环境以ROS软件包形式实现。我们采用ROS启动文件在Gazebo中将虚拟世界作为ROS节点进行部署，并为其填充模型。目前，该虚拟世界可进行模型填充。

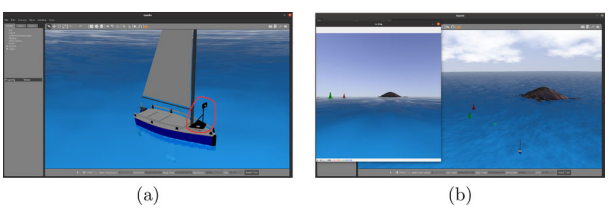



**图15**弓形摄像机：(a) 摄像机位置示意图。(b) 摄像机提供的场景示例

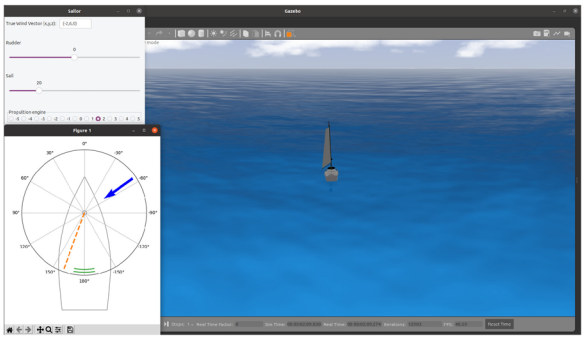


**Fig. 16** The figure shows some human interfaces of Yara. The GUI (Graphical User Interface) to enable the E-Boat manual control is on the top right. Below the controller GUI, we find a HUD displaying wind sensor data (blue arrow) and boom and propeller status. The orange dashed line represents the boom position, and the green solid lines represent the propeller speed

Using by four model types: the E-Boat, signal buoys, the ocean, and an island. The E-Boat and signal buoys are 3D models created with the help of Onshape (https://www.onshape.com/en/). The E-Boat characteristics, actuators, sensors, and physical properties are set using URDF and SDF description files. The ocean model implements the sea surface simulation using OpenGL shaders and an implementation of the Gerstner wave model [28, 36–39].

Figure 17 depicts the overall structure of our virtual sailing environment. We wrote or adapted plugins to implement some physics, boat controls, sensor arrays, and environmental control. These plugins are executed within the simulation loop in which Gazebo physics engine computes the new state of the models populating the virtual world. Figure 18 illustrates the simulation looping in a simplified way. We detail these plugins in the following sections.

### 4.2.1 Physics Plugins

Some of the physics used in the simulation are not defined within the Gazebo's physics engine. This additional physics is implemented as plugins written in C++ and compiled within the ROS package. They are: (i) the hydrodynamic plugin, (ii) the wave plugin which implements the ocean wave model responsible for the sea surface movement [28, 37–39], (iii) and the foil dynamic which implements the lift and drag forces acting on the sail, the rudder, and the keel. In Yara OVE, we combine the hydrodynamic and wave plugins with our original foil dynamic plugin to extend the Gazebo physics engine to create suitable physics to simulate sailing vessels.

There are two physics plugins to add hydrodynamic, hydrostatic, and aerodynamic forces to the Gazebo simulation and one more for ocean waves. The hydrodynamic plugin was adapted from the Virtual RobotX Challenge (https://github.com/osrf/vrx), adding the 6-DOF maneuvering model to the E-Boat simulation. The hydrodynamic force is computed based on the physical characteristics of the E-Boast, like added mass and drag coefficients. On the other hand, the hydrostatic force computation considers the sea surface state as defined by the wave plugin.

The foil dynamic plugin computes the lift and drag forces acting on the sail, rudder, and keel. These forces are computed at each physics engine time step following Eqs. 4 and 5. For the sail, the fluid velocity, represented by $V_A$ is the addition of the negative E-Boat velocity and the environment wind velocity (see Fig. 5). For the keel and the rudder,

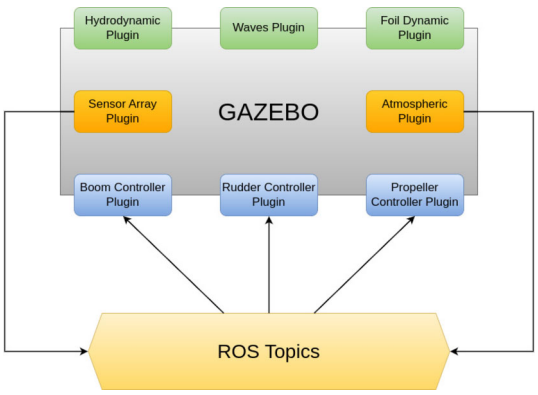


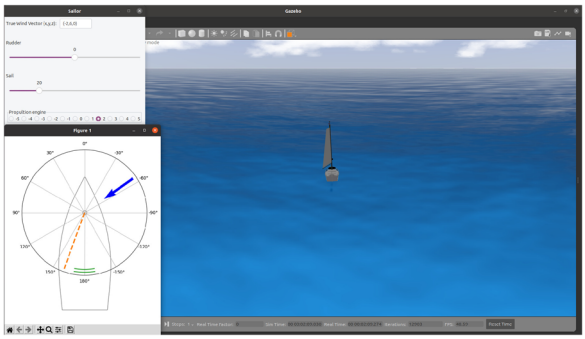

**Fig. 17** Overall system architecture. See Sec. 4.2 introduction for a description of the components




**Fig. 18** Simplistic visualization of simulation looping

**图16**该图展示了一些
Yara的人机界面。图形用户界面（GUI）

用于启用E-Boat手册
控制按钮位于右上角。
在控制器图形用户界面（GUI）下方，我们
寻找显示风速的平视显示器（HUD）
传感器数据（蓝色箭头）与
鼓风机与螺旋桨状态。
橙色虚线表示
看涨头寸与看涨信号
实线表示
螺旋桨转速



Using

通过四种模型类型：E艇、信号浮标、海洋和岛屿。E艇与信号浮标是借助Onshape（https://www.onshape./en/）创建的三维模型。E艇的特性参数、执行器、传感器及物理属性均通过 URDF 和SDF描述文件进行设定。海洋模型采用OpenGL着色器实现海面模拟，并基于Gerstner波浪模型进行建模[28, 36–39]。

图17展示了我们虚拟航海环境的整体架构。我们编写或修改了插件来实现物理模拟、船舶操控系统、传感器阵列及环境控制功能。这些插件在仿真循环中运行，其中Gazebo物理引擎负责计算虚拟世界中各模型的新状态。图18以简化形式展示了仿真循环流程。相关插件的具体实现细节将在后续章节中详细说明。

### 4.2.1 物理插件

模拟过程中使用的一些物理模型并未在Gazebo物理引擎中预先定义。这些额外物理模型通过C++编写的插件实现，并集成在ROS软件包中。具体包括：(i) 流体动力学插件，(ii) 波浪插件（负责实现海面运动的海洋波浪模型[28, 37–39]），(iii) 桨叶动力学插件（用于模拟作用于帆体、舵面及龙骨上的升力与阻力）。在Yara

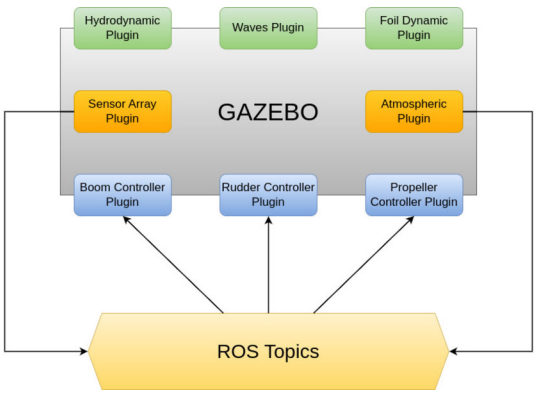



**图17**系统整体架构。组件描述详见第4.2节引言部分

OVE系统中，我们通过整合流体动力学插件、波浪插件与自主研发的桨叶动力学插件，对Gazebo物理引擎进行扩展升级，从而构建出适用于帆船模拟的完整物理模型体系。

Gazebo仿真系统配备了两个物理插件，分别用于添加流体动力学、流体静力学和空气动力学作用力，另有一个专门模拟海浪的插件。流体动力学插件改编自Virtual RobotX挑战赛（https://github.com/osrf/vrx），为E-Boat仿真系统新增了六 DOF 机动模型。流体动力学作用力的计算基于E-Boat的物理特性参数，如附加质量和阻力系数；而流体静力学作用力的计算则根据波浪插件定义的海面状态进行。

箔片动力学插件计算作用于帆、舵和龙骨上的升力与阻力。这些力根据公式4和5在每个物理引擎时间步长进行计算。对于帆而言，流体速度$VA$由负E-Boat速度与环境风速之和表示（见图5）。对于龙骨和舵，




**图18**模拟循环过程的简化可视化示意图

the fluid velocity is the addition of the negative velocity of the E-Boat and the natural fluid velocity caused by natural water currents. Figure 19 shows the vector operation used to compute the apparent fluid velocity ($V_A$). In the figure, $V_A$ is the fluid apparent velocity (called Inflow in Fig. 7), $V_T$ in the natural fluid velocity (wind true speed/water current or wave motion), $u$ is the surge velocity, $v$ is the sway velocity, and $V_B$ is the boat velocity. Therefore, the vector operation can be written in the following way: $\overrightarrow{V_A} = \overrightarrow{V_T} - \overrightarrow{V_B}$.

The wave plugin was also adapted from the Virtual RobotX Challenge. This plugin generates a wave field using a summation of Gerstner waves from a horizontal reference location $\boldsymbol{P_0} = (x_0, y_0)$ with no vertical displacement $z = 0$. The wave field is described by a horizontal ($\boldsymbol{P}$) and vertical ($z$) displacement related to the reference location $\boldsymbol{P_0}$ as:

$$\boldsymbol{P} = \boldsymbol{P_0} - \sum_{i=1}^{N} q_i A_i \sin(\boldsymbol{k_i} \cdot \boldsymbol{P_0} - \omega_i t + \phi_i) \tag{6}$$

$$z = \sum_{i=1}^{N} A_i \cos(\boldsymbol{k_i} \cdot \boldsymbol{P_0} - \omega_i t + \phi_i) \tag{7}$$

where $q_i$ is the steepness, $A_i$ is the amplitude, $\boldsymbol{k_i}$ is the wave vector, $\omega_i$ is the angular frequency, and $\phi_i$ is the wave phase.

#### 4.2.2 Control and Sensors Plugins

The control and sensor plugins are responsible for replicating the system that manages the E-Boat actuators and sensors. The control plugins are subscribers of three ROS topics, one for each actuator: boom, rudder, and propeller. At each time step of the physics engine, these plugins check for new commands and apply the required changes to the position of the actuators, as discussed in Section 4.1.1.

The Sensor Array plugin collects and transmits data from different sources within the E-Boat. It publishes all data in a ROS topic every four seconds, but the user can change the publishing frequency. The data vector is composed of distance to the current waypoint, trajectory angle, linear velocity, apparent wind speed, apparent wind angle, current boom angle, current rudder angle, current propeller speed, and roll angle.

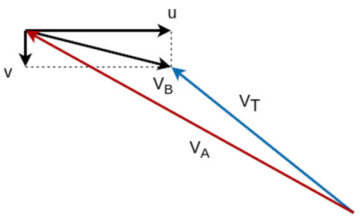



**Fig. 19** Diagram depicting the vector operation to compute the apparent fluid velocity $F_{ap}$

#### 4.2.3 Atmospheric Control Plugin

Usually, environmental conditions such as wind speed and direction, wave height and frequency, and fog are set when the simulation is launched. The atmospheric plugin was designed to allow users to change some environmental parameters during the execution. At the moment, we can only manipulate the wind speed and direction. However, we plan for the users to be able to include fog and manipulate the light conditions at any time. The user will also be able to control the ocean wave height and frequency, for example, and whether the wind speed and direction will automatically define the wave parameters.

### 4.3 Reinforcement Learning APIs

In terms of implementation, the standard approach for training and evaluating RL agents is by using an environment known as Gym [40]. A Gym environment is essentially an API that lets the learning agent interact with its environment. The environment may be a piece of software or some physical device. In our case, the environment is a software simulation of the sea and wind conditions encountered by the E-Boat. This API defines a few primitive calls to initialize the environment, get the current state/observation from the environment, specify the action to perform, and obtain three pieces of information. This information concerns the new state/observation, the return, and a flag indicating whether the task has been completed. The Gym is migrating to a new API named Gymnasium [11]. Essentially, Gymnasium is an upgraded version of the Gym to cope with some weaknesses. As Gymnasium is the current supported API to support Gym environments, we chose it so the environment of the E-Boat could provide resources for RL training end evaluation.

Making a Gymnasium environment with Gazebo and ROS is not a direct approach. We must consider the ROS messages, topics, communication delays, environment reset, actuators reset, launch files, etc. Gym-Gazebo [41] is based on OpenAI Gym and provides several tools for conducting the experiments. As we chose to work with Gymnasium, using Gym-Gazebo is not viable. Therefore, we developed our own Gymnasium-Gazebo environment class.

Once we have our Gymnasium environment, we must connect an RL agent to be trained and evaluated. There are several libraries for training and evaluating RL agents like RL-Berry [42], RL_Coach [43], MushroomRL [44], CleanRL [45], Tensorforce [46], and KerasRL [47]. In the first phase of this work, we have chosen to use the Stable-Baselines3 library [10], which is largely used in the reinforcement learning community.

流体速度是E艇的负速度与自然水流引起的自然流体速度之和。图19展示了用于计算表观流体速度（$VA$）的矢量运算。其中，$VA$为流体表观速度（图7中称为流入速度），$VT$为自然流体速度（风真速/水流或波浪运动），$u$为涌浪速度，$v$为横摇速度，$VB$为艇体速度。因此，该矢量运算可以表示为：$\vec{VA}=\vec{VT}-\vec{VB}$。

波形插件同样改编自Virtual RobotX挑战赛。该插件通过从水平参考位置$\boldsymbol{P_0}=(x0, y0)$处生成Gerstner波的叠加来构建波场，且无垂直位移$z=0$。波场由与参考位置$\boldsymbol{P_0}$相关的水平位移($\boldsymbol{P}$)和垂直位移($z$)描述，具体表达式为：

$$\boldsymbol{P} = \boldsymbol{P_0} - \sum_{i=1}^{N} q_i A_i \sin(\boldsymbol{k_i} \cdot \boldsymbol{P_0} - \omega_i t + \phi_i) \tag{6}$$

$$Nz = \sum_{i=1} A_i \cos(\boldsymbol{k_i} \cdot \boldsymbol{P_0} - \omega_i t + \phi_i) \tag{7}$$

其中$q_i$为斜率，$A_i$为振幅，$\boldsymbol{ki}$为波矢，$\boldsymbol{wi}$为角频率，$\boldsymbol{\varphi i}$为波相位。

#### 4.2.2 控制与传感器插件

控制插件与传感器插件负责复现管理电动船执行器及传感器的系统架构。控制插件订阅了三个ROS话题，分别对应三个执行器：横杆、舵机和螺旋桨。在物理引擎的每个时间步长中，这些插件会实时检测新指令，并根据第4.1.1节所述内容，对执行器位置进行相应调整。

传感器阵列插件负责采集并传输E-Boat船体内的多源数据。系统每四秒通过ROS话题发布完整数据集，用户可自定义更新频率。数据向量包含以下参数：当前位置航点距离、航迹角、线速度、表观风速、表观风向角、当前横杆角度、当前舵角、当前螺旋桨转速以及横摇角。

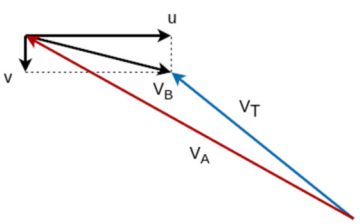



**图19**描述计算表观流体速度$F_{ap}$的矢量运算示意图

#### 4.2.3 大气控制插件

通常在启动模拟时，需要预先设定风速与风向、波高与波频以及雾量等环境参数。大气插件的设计初衷是让用户在运行过程中能够调整部分环境参数。目前我们仅支持风速与风向的调节功能，但计划未来将新增雾量参数，并实现光照条件的实时操控。用户还将能够自主控制海浪波高与波频参数，例如设定风速与风向是否自动定义波浪参数。

### 4.3 强化学习API

在实施层面，训练和评估强化学习智能体的标准方法是使用名为Gym的环境[40]。Gym环境本质上是一个允许学习智能体与环境交互的API接口，该环境可以是软件程序或物理设备。在本案例中，环境采用模拟E艇所处海况与风况的软件模型。该API定义了若干基础调用方法：用于初始化环境、获取当前状态/观测数据、指定执行动作，并获取三项关键信息——新状态/观测值、任务回报值以及任务完成标志。Gym环境正在向名为Gymnasium的新API过渡[11]。Gymnasium本质上是对Gym的升级版本，旨在解决原有版本存在的缺陷。由于Gymnasium是当前支持Gym环境的官方API，我们选择采用该版本以确保E艇环境能够为强化学习训练与评估提供必要资源。

使用Gazebo和ROS构建体育馆环境并非直接可行方案。我们需要综合考虑ROS消息传递机制、主题系统配置、通信延迟控制、环境参数重置、执行器复位设置以及启动文件配置等关键要素。Gym-Gazebo[41]基于OpenAI Gym框架开发，虽提供多种实验工具，但鉴于我们选择采用Gymnasium平台，直接使用Gym-Gazebo存在技术局限性。为此，我们自主研发了体育馆-Gazebo环境集成类。

一旦我们构建好体育馆环境，就必须连接一个强化学习（RL）智能体进行训练和评估。目前存在多个用于训练和评估RL智能体的库，例如RL-Berry[42]、RL_Coach[43]、MushroomRL[44]、CleanRL[45]、Tensorforce[46]以及KerasRL[47]。在本研究的第一阶段，我们选择使用Stable-Baselines3库[10]，该库在强化学习领域被广泛采用。

**Table 2** Yara's Gymnasium environments

| environment | electric propeller | object detection |
|---|---|---|
| Eboat62-v0 | NO | NO |
| Eboat63-v0 | YES | NO |
| Eboat93-v0 | YES | YES |

We provide, within Yara, three Gymnasium environments to train agents using deep RL algorithms. The solution is in Python, and the base class can be easily used to develop new training environments. Table 2 depicts the main characteristics of each one of Yara's Gymnasium environments. Eboat62-v0 enables training in a pure sailing boat, in which the electric propeller is not available. In Eboat63-v0, the propeller is available, but the designed return function penalizes its use. Eboat93-v0 implements object detection in a 120-degree arc ahead of the boat. These environments provide an easy way to train new decision policies by changing the return function, the neural network architecture, the wind direction/speed sampling, etc.

## 5 Experimental Results

The main objective of our environment is to provide a set of tools to enable the development and testing of RL agents in a simulated environment with reliable physics. In this section, we present some experiments that describe the EBoat's maneuverability (Section 5.1) and some autonomous navigation experiments using RL agents and a PID-based controller (Section 5.2).

### 5.1 E-Boat response in maneuvers

To validate the E-Boat maneuverability, we performed a qualitative evaluation of different behaviors as shown in Figs. 20, 21, 22, 23, and 24.

We can observe a clear trend by examining Fig. 20a, which illustrates the relationship between velocity and distance. As the intensity of propulsion increases from 1 to 5, the E-Boat experiences a gradual acceleration, covering larger distances within the same time intervals.

Figure 20b depicts the relationship between velocity and time, where we can discern a similar pattern. As the E-Boat's propulsion intensity increases, its velocity rises steadily. The velocity stabilization happens after 10 seconds at the highest propeller speed while taking 20 seconds for a slow-pace propeller. This observation further reinforces the understanding of the effects of varying propulsive forces on the boat speed.

From a qualitative perspective, the graphs confirm the intu itive understanding that breaking the initial inertia requires an initial burst of energy. Subsequently, maintaining momentum becomes a delicate balance between the energy consumed by hydrodynamic and aerodynamic forces, including those from the atmosphere (assuming wind speed is zero). This interplay between propulsive and resistive forces is a familiar concept to experienced sailors, who rely on such knowledge to optimize their performance on the water.

Figure 21 depicts the coordinates of a stationary boat that starts to propel itself using its propeller at different intensities. In Fig. (21a), the boat makes a curve to starboard with a rudder angle of 60°, while in Fig. (21b), it makes a curve to port with a rudder angle of 60°. It is possible to observe that with the highest propeller intensity (violet plot), the boat develops a complete circle with a radius close to its size, which is an expected behavior.

When analyzing the results, it becomes evident that the boat's turning radius decreases as propulsion velocity increases. This behavior aligns with the hydrodynamics of the rudder, where lower water velocities result in reduced maneuvering capabilities. These observations highlight the relationship between propulsion velocity and a sailboat's turning abilities, enhancing our understanding of boat maneuvering dynamics.

Figure 22 shows the coordinates of a stationary boat propelled solely by the electric propeller. The boat successfully

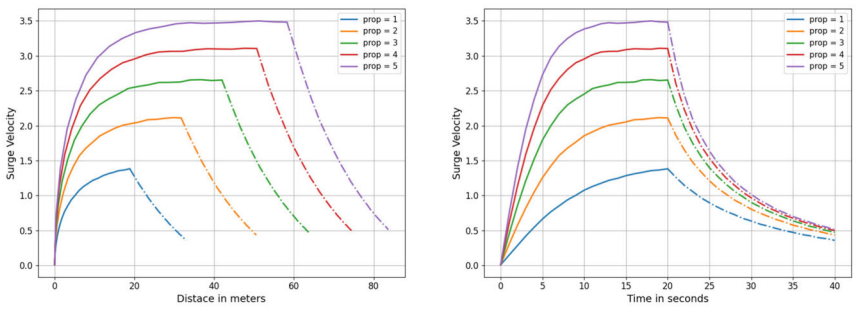



**Fig. 20** E-Boat acceleration for different propeller speeds

**表2** Yara体育馆环境

| 环境 | 电推进器 | 目标检测 |
|---|---|---|
| Eboat62-v0 | 不 | 不 |
| Eboat63-v0 | 是 | 不 |
| Eboat93-v0 | 是 | 是 |

我们在Yara平台内提供了三种体育馆环境，用于基于深度强化学习算法训练智能体。该解决方案采用Python语言开发，基础类模块可轻松扩展以构建新型训练环境。表2详细列出了Yara体育馆环境的主要特性：Eboat62-v0环境模拟纯传统帆船训练场景，未配备电动螺旋桨；Eboat63-v0环境虽保留螺旋桨功能，但通过设计回报函数对螺旋桨使用进行惩罚性约束；Eboat93-v0环境则实现了对船舶前方120度弧形区域的物体检测功能。这些环境通过灵活调整回报函数、神经网络架构、风向/风速采样参数等参数，为新决策策略的训练提供了便捷途径。

## 5 实验结果

我们开发该环境的主要目标是提供一套工具，用于在具有可靠物理特性的模拟环境中开发和测试强化学习（RL）智能体。本节将展示一系列实验：包括描述EBoat机动性能的实验（第5.1节），以及利用强化学习智能体与基于PID控制器进行自主导航的实验（第5.2节）。

### 5.1 机动过程中E艇的反应

为验证E-Boat的机动性，我们对不同行为进行了定性评估，如图20、21、22、23和24所示。

通过分析图20a可以清晰观察到速度与距离之间的关系趋势。当推进强度从1增至5时，E型船呈现渐进式加速特性，在相同时间间隔内能覆盖更长距离。

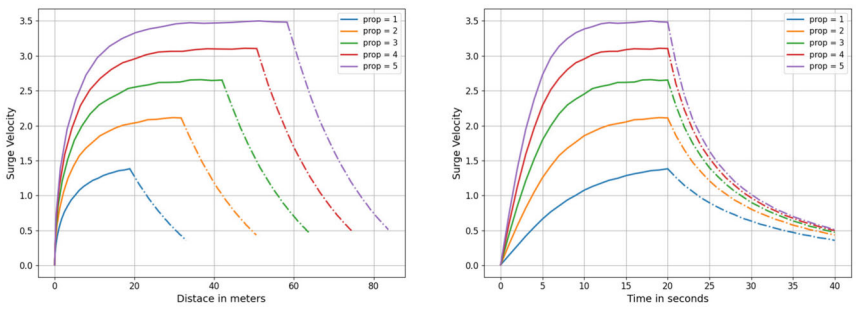
图20b展示了速度与时间的关系曲线，从中可以清晰观察到相似的规律。随着E型船推进强度的提升，其速度呈现稳步增长趋势。当螺旋桨转速达到最高值时，速度稳定现象在10秒后显现；而采用低速推进模式时，这一过程则需要20秒时间。该观测结果进一步印证了推进力变化对船舶速度产生的影响机制。

从定性分析的角度来看，这些图表印证了我们的直觉认知：要打破初始惯性状态，必须先释放初始能量。随后，维持动量状态就成为水动力与气动力（包括大气阻力，假设风速为零）所消耗能量之间的微妙平衡。推进力与阻力之间的这种相互作用机制，正是经验丰富的航海者所熟知的概念——他们正是凭借这类知识来优化水上航行表现。

图21展示了静止船只在不同螺旋桨强度下开始自主推进时的坐标分布。在图(21a)中，船只以60度舵角向右舷划行；而在图(21b)中，船只以相同舵角向左舷划行。可以观察到，在螺旋桨强度最高时（紫色曲线），船只会形成半径接近其自身尺寸的完整圆周运动轨迹，这与预期行为一致。

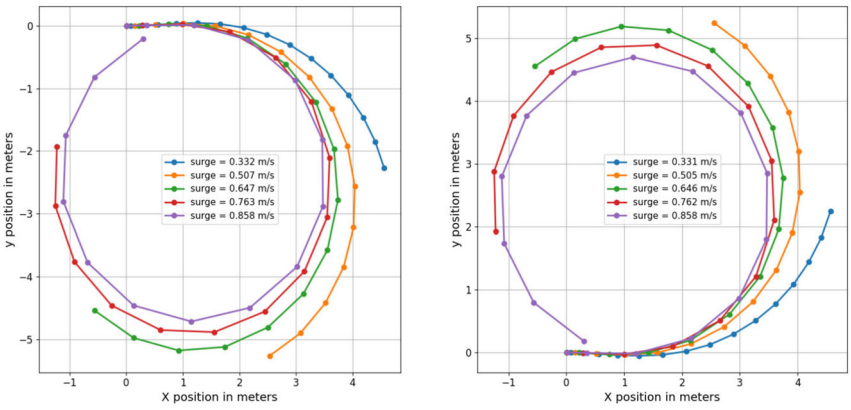
分析结果表明，随着推进速度的提升，帆船的转向半径会逐渐减小。这一现象与舵的流体力学特性相符——水流速度越低，船舶的操控性能就越差。这些观测数据揭示了推进速度与帆船转向能力之间的内在关联，为深入理解船舶操控动力学机制提供了重要依据。

图22展示了仅由电动螺旋桨驱动的静止船只坐标。该船只成功




(a) (b)

**图20**不同螺旋桨转速下的E艇加速度




(a) (b)

**Fig. 21** E-Boat turning under different velocities. The rudder angle is fixed in $60^o/-60^o$, and we tried five runs with different propeller speeds. Each run lasts fifteen seconds. The position and surge velocity data are collected every second

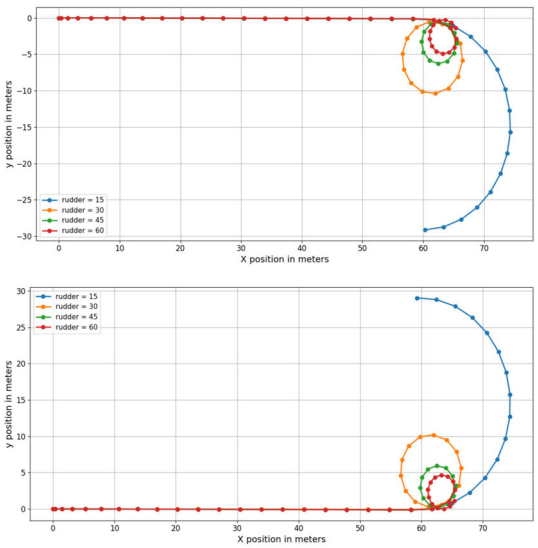



**Fig. 22** E-Boat turning maneuver executed after the virtual boat reaches maximum velocity. The propeller is set to its maximum power, and the boat's velocity right before the turning is around 3.5 m/s

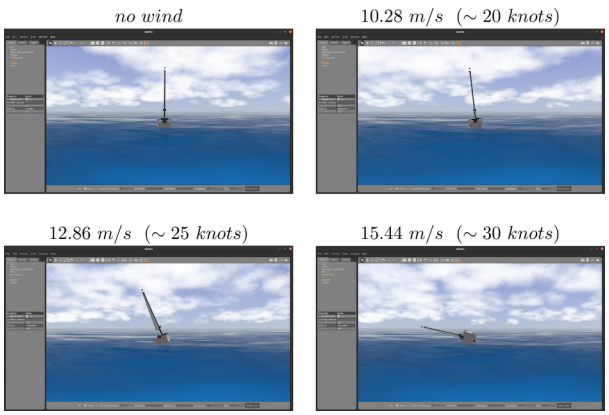
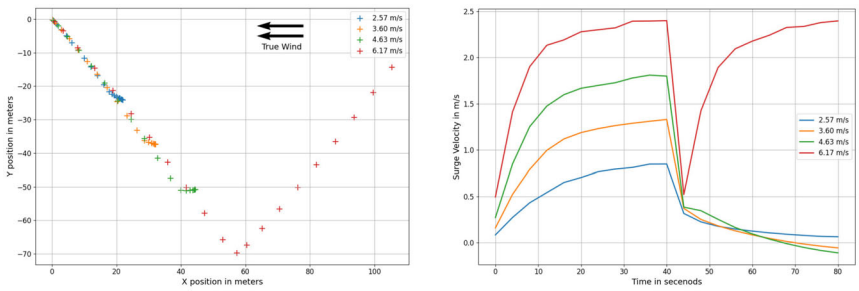
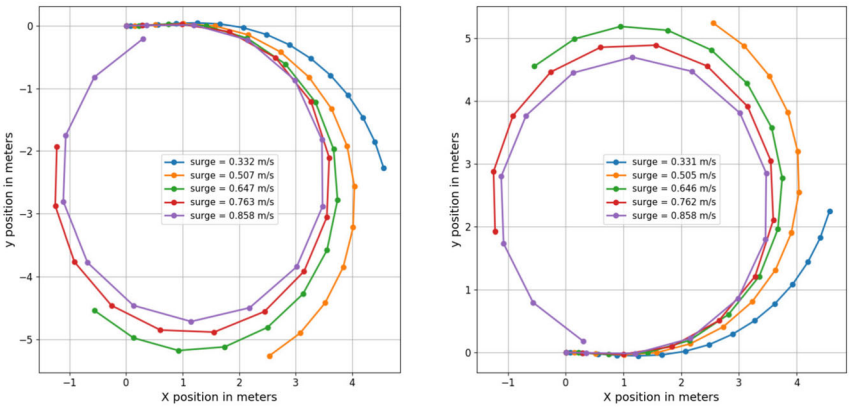



**Fig. 24** The listening of the E-Boat under the influence of different wind speeds. The wind blows perpendicular to the sail plan. We can observe the $Y$ component of the wind velocity vector in each picture in m/s

**Fig. 23** Tack maneuver under different wind speeds. The legend depicts the true wind speed. Figure (a) shows the boat's position, while Figure (b) shows the boat's surge velocity. The position and velocity values were collected every four seconds




(a) (b)




(a) (b)

**图21** E型船在不同速度下的转向情况。舵角固定为60°±60 °，我们尝试了五次不同螺旋桨转速下的运行测试，每次测试持续十五秒。位置数据与涌浪速度数据每秒采集一次。

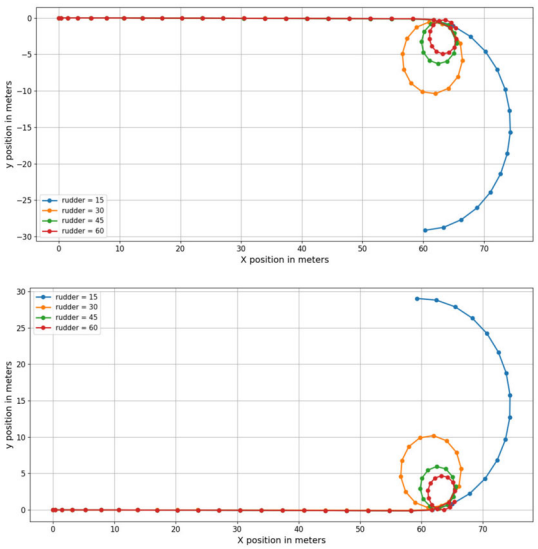



**图22**虚拟船达到最大速度后执行的E型船转向操作。螺旋桨被设置为其最大功率，且转向前船速约为3.5 m/s。

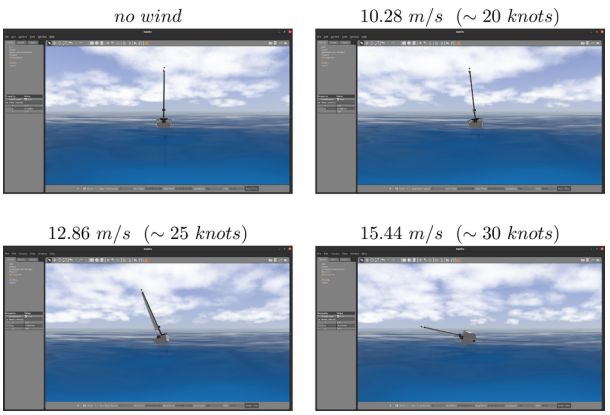
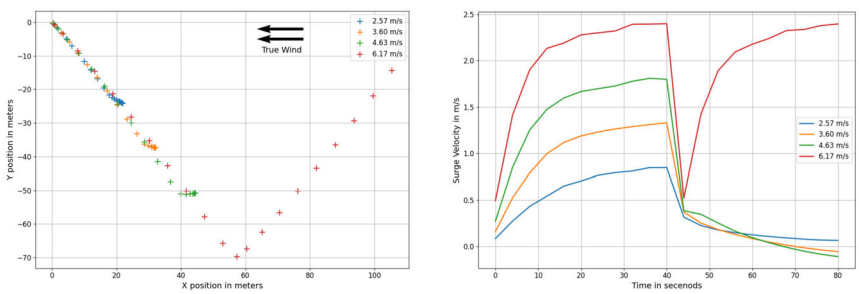



**图24**不同风速下E艇的听觉效果。风垂直于帆面吹来。我们可以在每张图片中观察到风速矢量的$Y$分量，单位为m/s。

**图 23** 不同风速下的追风机动。图例标注了真实风速。图(a)显示船舶位置，图(b)显示船舶涌浪速度。位置与速度数据每四秒采集一次。




(a) (b)

performs curves to both starboard (top) and port (bottom), demonstrating satisfactory maneuverability under the given conditions. These results provide valuable insights into the boat's maneuverability and the hydrodynamics of its rudder. They demonstrate the effectiveness of the boat's steering mechanism in executing curves.

Figure 23a depicts the coordinates of a stationary boat propelled solely by the wind with various tested velocities. As the simulation approaches approximately 40 seconds, the boat attempts to perform a maneuver known as a tack. Figure (23b) displays the boat velocity over time. Although the first graph implies it, the second graph shows that the boat can only execute the tack maneuver at 2.3 m/s or higher speeds. This may depend on different boat characteristics, but it is well known that it cannot perform any maneuver if the boat has a small velocity course. However, these results do not align well with the qualitative expectations. It was anticipated that, under real conditions, the boat would still be able to perform the tack maneuver at speeds as low as 1.5 m/s. Nevertheless, the obtained results are still closer to the expectations.

Figure 24 showcases the effects of wind intensity on the sail and keel of the sailboat. Four wind intensities are iterated: No wind, 20, 25, and 30 knots. As a result, we can observe how the wind impacts the sail and the sailboat's keel. These observations align with the expected behavior, considering that real sailboats often have a low keel and a significant ballast relative to the total weight of the vessel. This configuration results in a low center of gravity, minimizing the inclination caused by the demonstrated winds. In sailing, 30 knots is considered dangerous and may cause a situation similar to the one presented in Fig. 24, depending on the weight and keel.

## 5.2 Autonomous Navigation

Control tasks for a sailing robot present unique challenges not encountered with traditional motor-powered USVs, such as performing a tacking maneuver while sailing upwind or adjusting the jib when sailing downwind. Additionally, ensuring the boat's safety by adjusting the course and sail position to avoid any potentially dangerous situations is crucial (see Fig. 24).

In this section, we describe some experiments performed with three different control agents: a PID-based controller, a police-based controller trained with Proximal Policy Optimization (PPO) [48], and a police-based controller trained with Soft Actor-Critic (SAC) [49]. Our aim is not to present a performance comparison but rather to showcase the implementation of RL training and evaluation using Yara and to demonstrate how ambient factors such as waves and wind can impact mission execution.

### 5.2.1 PID Controler

A PID control can be used for a simple control of the boat's movement. To reach a desired waypoint in the platform workspace, a simple strategy is to have two decoupled PID controllers, one for the heading and the other for the boat's forward speed (surge). The PID consists of calculating the error of the control variable, heading or speed in this case, that is, the error between a reference input $x_d$ and the current value $x$ of the] variable, as shown in Eq. 8. A higher-level navigation system sets the reference input.

$$e_x = x_d - x \tag{8}$$

The next step is to apply the proportional, integral, and derivative gains to the error, as shown in Eq. 9, to find the appropriate commands for the actuators, in this case, the rudder angle and motor speed.

$$\delta_r = K_p e_x(t) + K_i \int e_x(t)\, dt + K_d \frac{de_x}{dt} \tag{9}$$

For sailing robots, the heading control is less robust to lateral drifts caused by both the water current and the wind, and in some situations, a course control, where the velocity vector is the control variable.

### 5.2.2 RL Agents

We trained two RL agents that are available within Yara. To begin with, we took a straightforward approach for the initial environment test. As a navigation path can be divided into multiple waypoints, we trained the agent to reach a single waypoint located 100 meters away. Once the agent masters the skill of reaching a waypoint, it can progress to the next location in the navigation path simply by changing the waypoint each time the sailing robot reaches one.

We chose PPO and SAC, which are well-known to fit such settings as the observation space and the continuous action space.

For this experiment, the observational space is composed of five parameters: the current distance from the waypoint; the angle between the E-Boat forward direction and the position of the waypoint; surge velocity (see Fig. 1); apparent wind speed; the apparent wind angle. The action space is composed by all three actuators within EBoat: rudder, boom, and propeller.

该船在右舷（上图）和左舷（下图）均能完成曲线航行，表明其在给定条件下具备良好的操纵性。这些结果为船舶操纵性能及舵机流体力学特性提供了重要依据，同时验证了船舶操舵系统在曲线航行中的有效性。

图23a展示了仅靠风力驱动的静止船只在不同测试速度下的坐标分布。当模拟接近约40秒时，该船尝试执行一种称为横移的机动动作。图(23b)显示了船只随时间变化的速度曲线。尽管第一张图表暗示了这一点，但第二张图表表明船只仅能在2.3 m/s或更高速度下执行横移机动。这可能取决于不同船只的特性，但众所周知，若船只速度过低则无法执行任何机动动作。然而这些结果与定性预期并不完全吻合。原本预计在实际条件下，船只仍能在低至1.5 m/s的速度下完成横移机动。尽管如此，所得结果仍更接近预期值。

图24展示了风力强度对帆船帆面及龙骨结构的影响。实验设置了四种风速等级：无风、20节、25节和30节。通过对比分析，我们可以清晰观察到风力对帆面及龙骨结构的具体作用机制。这些观测结果与理论预期高度吻合——实际帆船通常采用低龙骨设计，并配备占船体总重量较大比例的压舱物。这种结构布局使得重心位置较低，有效降低了受测试风力作用时的倾斜幅度。在航海实践中，30节风速属于危险级别，根据船舶载重和龙骨设计差异，可能导致类似图24所示的航行风险。

## 5.2 自主导航

对于航行机器人而言，其控制任务面临传统电动水下航行器（USV）所不具备的独特挑战，例如在迎风航行时执行转向操作，或在顺风航行时调整前帆姿态。此外，通过调整航向与帆位来规避潜在危险情境，从而确保航行器安全至关重要（参见图24）。

本节将介绍基于三种不同控制策略开展的实验：基于PID控制的控制器、采用近端策略优化（PPO）算法训练的警用控制器[48]，以及运用软体演员-评论家（SAC）算法训练的警用控制器[49]。我们并非旨在进行性能对比分析，而是通过Yara平台展示强化学习训练与评估的实现过程，并揭示海浪、风力等环境因素对任务执行的影响机制。

### 5.2.1 PID控制器

PID控制可用于对船舶运动进行简单控制。为了在平台工作空间中到达目标航点，一种简单策略是采用两个解耦的PID控制器，一个用于航向，另一个用于船舶前进速度（涌浪）。PID控制的核心在于计算控制变量（此处为航向或速度）的误差，即参考输入$xd$与变量当前值$x$之间的误差，如等式8所示。高级导航系统负责设定参考输入。

$$e_x = x_d - x \tag{8}$$

下一步是将比例、积分和微分增益应用于误差，如等式9所示，以找到执行器的适当指令，在此情况下为舵角和电机速度。

$$\delta_r = K_p e_x(t) + K_i \int e_x(t)\, dt + K_d \frac{de_x}{dt} \tag{9}$$

对于航行机器人而言，航向控制对由水流和风力引起的横向漂移具有较低鲁棒性，在某些情况下，需要采用航向控制策略，其中速度矢量作为控制变量。

### 5.2.2 RL代理

我们训练了Yara平台中可用的两个强化学习（RL）智能体。在初始环境测试阶段，我们采用了基础测试方案：由于导航路径可划分为多个航点，我们训练智能体完成100米外单个航点的抵达任务。当智能体掌握航点抵达技能后，即可通过每次航行机器人到达新航点时切换目标航点的方式，实现对导航路径中后续位置的连续探索。

我们选择了PPO和SAC，这两种方法在适应观察空间和连续动作空间等场景方面具有广泛认可度。

本实验的观测空间包含五个参数：当前与航点的距离、E-Boat前进方向与航点位置之间的夹角、涌浪速度（见图1）、表观风速以及表观风向角。动作空间则由E-Boat上的三个执行器构成：舵机、吊杆和螺旋桨。

As the waypoint position and wind conditions could change from mission to mission, the agent should learn how to behave in different initial conditions. Using a uniform distribution, we randomly sample the wind speed, wind direction, and boat orientation for every environment reset. The wind speed is sampled for the interval $[2, 6]$, the wind direction from $(-180, 180]$, and the boat orientation from $(-180, 180]$.

The return function is described in Algorithm 2. All actions that decrease the distance from the goal are associated with a positive return, while those increasing the distance are associated with a negative return, hence penalized. It is a desired behavior that the agent uses the electric propeller only in situations where the wind force is not enough to push the boat forward or when a maneuver against the wind is needed. Therefore, returns earned by the agent are decreased by a value proportional to the current propeller speed ($PS_t$).

**Algorithm 2** The function that computes the immediate return $r$ at time $t$, $r_t$.

$\Delta D \Leftarrow D_{t-1} - D_t$ ▷ $D_t$ is the distance from the goal in time $t$.

▷ $D_{t-1}$ is the distance from the goal in time $t-1$.

$D_{max} = 1.25 * D_0$ ▷ $D_0$ is the distance from the goal in the initial state,

$t = 0$.

**if** $D_t < 5$ **then**

  $r_t \Leftarrow 1$

**else if** $D_t > D_{max}$ **then**

  $r_t \Leftarrow -1$

**else if** $\Delta D > 0$ **then**

  $r_t \Leftarrow \frac{\Delta D}{D_{max}} * \left(1.0 - 0.9 * \frac{|PS_t|}{5.0}\right)$ ▷ $PS_t$ is the propeller speed at time $t$

**else**

  $r_t \Leftarrow \frac{\Delta D}{D_{max}} - 0.01 * |PS_t|$

**end if**

We trained a functional agent using two neural networks, each made of two hidden layers with thirty-two ReLU units each. All other PPO parameters were set to the standard values used in Stable-baselines3, except that `ent_coef` parameter was set to 0.001. The training length was $5.0 \times 10^5$ steps. We performed all our experiments training the agent on a laptop running Ubuntu 20.04 and powered by an $11^{th}$ Gen Intel Core i7-11800H (2.30GHz), 16GB DDR5 RAM, and an Nvidia RTX 3060 GPU. The whole training process took three hours and forty minutes.

In this preliminary experiment, we aim for a policy to enable the agent to reach the goal using the wind as the preferred propulsion, i.e., with little use of the propeller. Figure 25 shows the average surge velocity of the boat for different wind speeds. In the worst-case scenario, the E-Boat should travel with a surge velocity of about one meter per second. The expected return should be at least 1.6, and an episode should not exceed fifty steps. Figure 26a displays the average return per episode during training, while Fig. 26b displays the ratio between the episode length and the expected worst episode length (50 steps). The experimental data shows that, on average, the agent exceeded our expectations.

### 5.2.3 Seven Waypoints Mission

In this experiment, we evaluate the execution of a seven-waypoint mission. Throughout the mission, the EBoat must navigate under different wind conditions. For our evaluation, we built a scenario with a wind speed of approximately 6 m/s and relatively calm ocean waters.

The trajectory of the E-Boat during mission execution is shown in Fig. 27. The PPO policy and PID-based controller follow a very similar course. However, differences become apparent when we analyze the mission execution data presented in Fig. 28. The SAC policy does not perform as well as the other two, and to avoid obscuring other data, we have plotted the SAC data as a dotted line. Between 0 and 154 seconds, the performance achieved by PID and PPO is very similar when the E-Boat is sailing in running, broad reach, or beam reach (see Fig. 29). During close reach sailing (periods 154 to 193 and 306 to 368), PPO achieves higher surge velocities than PID. However, between 193 and 306 seconds, PID reaches higher speeds when sailing against the wind.

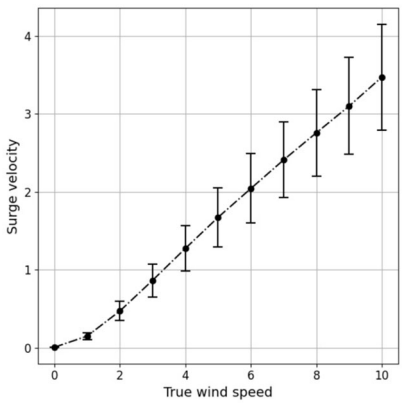



**Fig. 25** Average surge velocity as a function of the apparent wind speed. The values represent the velocity in a linear course averaged over different wind directions

由于航路点位置和风况可能因任务不同而变化，智能体需要学会在不同初始条件下采取相应行为。我们采用均匀分布法，在每次环境重置时随机采样风速、风向及船舶方位参数：风速取值区间为[2, 6]，风向取值范围为(−180,180]，船舶方位取值范围为(−180, 180]。

返回函数的具体实现详见算法2。所有能缩短与目标距离的操作都会获得正向回报，而增加距离的操作则会因负向回报而受到惩罚。智能体应遵循这样的行为准则：仅在风力不足以推动船只前进，或需要逆风操控时才使用电动螺旋桨。因此，智能体获得的回报值会根据当前螺旋桨转速（$PS_t$）按比例递减。

**算法2**计算时间$t$时即时回报$r$的函数，$r_t$。

1: $D_{\text{当}}$ $Dt_{-1} - Dt$口$Dt$表示时间$t$时与目标的距离。

2: 口$Dt_{-1}$表示在时间$t-1$时与目标的距离。

3:$D_{max}$= 1.25 $D_0$口$D_0$表示初始状态下到目标的距离，

4: $t = 0$.

5：**如果**$Dt$ <5**则**

6: $r_t \Leftarrow 1$

7：**else if** $Dt > Dmax$**且**

8: $r_t \Leftarrow -1$

9：**else if** $D$ >0**且**

10: $r_t \Leftarrow \frac{\Delta D}{D_{max}} * \left(1.0 - 0.9 * \frac{|PS_t|}{50}\right)$ 口 PSt 是时间t时的螺旋桨速度

11：**否则**

12: $r_t \Leftarrow \frac{\Delta D}{D_{max}} - 0.01 * |PS_t|$

13：**end if**

我们采用两个神经网络训练功能智能体，每个网络包含两个隐藏层，每层配置32个ReLU激活单元。除ent_coef参数设为0.001外，其余PPO参数均采用Stable-baselines3研究中的标准值。训练周期为$5.0\times10^5$步。所有实验均在搭载Ubuntu 20.04系统的笔记本电脑上完成，该设备配备第11代英特尔酷睿i7-11800H处理器（2.30GHz主频）、16GB DDR5内存及Nvidia RTX 3060显卡。整个训练过程耗时3小时40分钟。

在本次初步实验中，我们旨在设计一种策略，使智能体能够以风力作为主要推进方式实现目标，即尽量减少螺旋桨的使用。图25展示了不同风速下船只的平均涌浪速度。在最坏情况下，E型船的涌浪速度应维持在每秒约1米。预期回报率需达到至少1.6，且单次训练回合不应超过50步。图26a呈现了训练期间每回合的平均回报率，而图26b则展示了训练回合时长与预期最坏回合时长（50步）的比值。实验数据显示，智能体的表现平均而言超出了我们的预期。

### 5.2.3 七航点任务

在本实验中，我们评估了七点任务的执行情况。在整个任务过程中，EBoat必须在不同风况下进行导航。为进行评估，我们构建了一个风速约为6 m/s且海水相对平静的场景。

图27展示了E艇在执行任务过程中的运动轨迹。PPO策略与基于PID的控制器表现出高度相似的运行轨迹。然而，当我们分析图28所示的任务执行数据时，差异性逐渐显现。SAC策略的表现逊于其他两种方法，为避免干扰其他数据，我们以虚线形式呈现了SAC数据。在0至154秒期间，当E艇处于顺风航行、横风航行或横风顺风航行状态时，PID与PPO的性能表现极为接近（参见图29）。在近风航行阶段（154至193秒及306至368秒时段），PPO策略获得的涌浪速度高于PID策略。但在193至306秒期间，PID策略在逆风航行时展现出更高的速度表现。

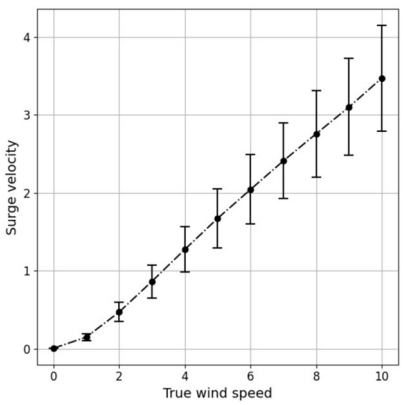



**图25**平均涌浪速度随表观风速的变化关系。数值表示沿直线路径在不同风向下的平均速度。

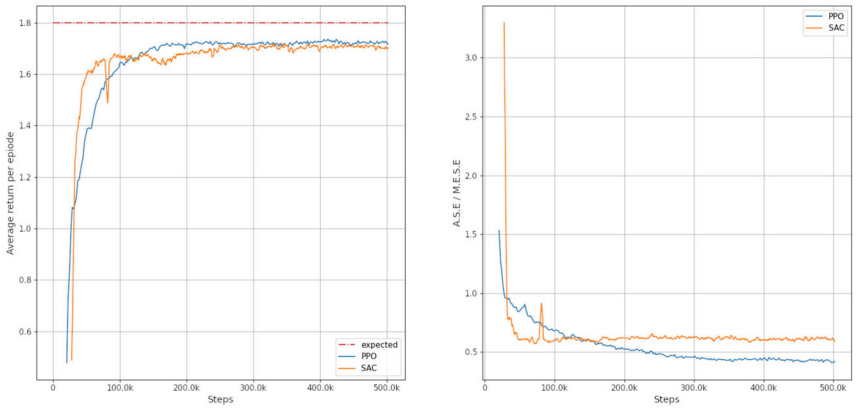



**Fig. 26** Training data. Figure (a) shows the agent's average return value per episode. Figure (b) shows the ratio between the average steps per episode (A.S.E.) and the maximum expected steps per episode (M.E.S.E.), which is equal to fifty

Figure 28d shows us that PID applies more power to the propeller than PPO, indicating that PPO achieves similar results using less energy. These results demonstrate how our environment can aid in the development, evaluation, and comparison of various approaches to implementing autonomous control for sailing robots.

## 6 Conclusion

There are not many open-source OVEs available, and only USVSim seems to offer a sailboat simulation. Different authors proposing new GNC solutions usually implement simplistic simulations or use simulations that do not represent the complex motion of a sailboat. Yara comes as tool to address this issue, offering different resources to make easy developing and testing GNC solutions with a reliable physics simulation. The digital twin E-Boat was modeled with a 6-DOF model parametrized to emulates a actual sailing robot. In this way, path-planning algorithms and other control solutions, such as Deel Reinforcement Learning (DRL), can be evaluated in different environmental conditions, with dynamic, static or no obstacles. As the use of DRL methods are growing in the literature, Yara facilitates training and testing of Reinforcement learning applications are enabled through Python classes that implement Gymnasium environments and integration with Stable-baselines3. We could not find other open-source OVE with a read-to-go integration with RL APIs.

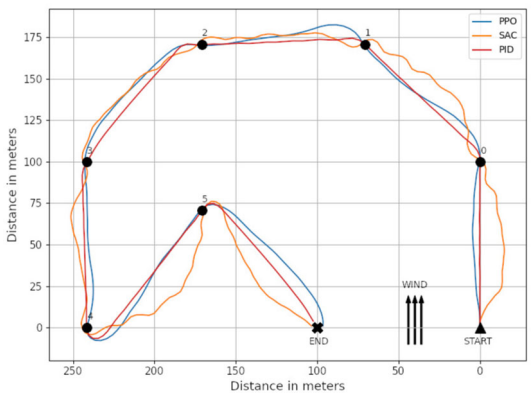



**Fig. 27** E-Boat's trajectory

Within Yara, we build the E-Boat, digital twin of real sailing robots built in Brazil for environmental monitoring. The physics models implemented within the E-Boat enable a reliable simulation of its motion under different ambient conditions, such as waves, water currents, and wind. The sail is modeled as a standard sail, not a rigid sail.

Our experiments demonstrate that E-Boat's simulated maneuverability and response of the sailing boats under different wind conditions closely resembled real-world behavior, showcasing the accuracy of the physics modeling implemented in Yara. Additionally, the integration with Gymnasium and Stable-baselines3 allowed us to train RL agents for autonomous GNC decision-making successfully. The experiments depicted in Section 5.2.3 demonstrate the practical applicability of Yara as a reliable tool for implementing and testing different GNC strategies and comparing them.

In future work, we will improve the wave model by creating an interface between it and the wind so the wind conditions modify the water surface conditions. We will also

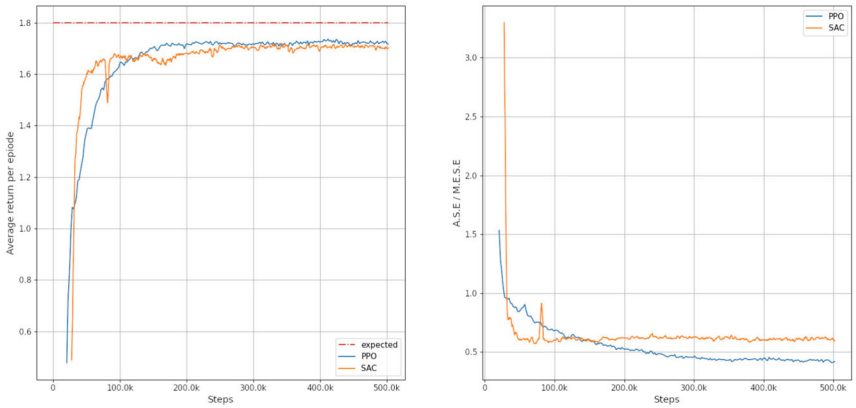



**图26**训练数据。图(a)显示了智能体每轮的平均回报值。图(b)展示了每轮平均步数（A.S.E.）与每轮最大期望步数（M.E.S.E.）的比值，该比值等于五十。

图28d显示，PID算法对螺旋桨的功率分配高于PPO算法，这表明PPO算法在能耗更低的情况下仍能取得相近效果。这些结果表明，我们的研究环境能够有效辅助帆船机器人自主控制技术的多种实施方案开发、评估与对比工作。

## 6 结论

目前可用的开源开放虚拟环境（OVE）并不多，其中只有USVSim似乎提供了帆船模拟功能。提出新型GNC 解决方案的开发者通常采用简化模拟方案，或使用无法真实反映帆船复杂运动特性的模拟方法。Yara作为针对这一问题的解决方案应运而生，通过提供多样化资源支持，使开发和测试 GNC 方案时能够获得可靠的物理模拟效果。其数字孪生体E-Boat采用六自由度 DOF 模型进行建模，并通过参数化设置模拟真实航行机器人。这种设计使得路径规划算法及其他控制方案（如深度强化学习 DRL）可在动态/静态/无障碍物等多种环境条件下进行评估。随着文献中 DRL 方法的应用日益广泛，Yara通过Python类实现健身房环境模拟及与Stable-baselines3的集成，为强化学习应用的训练与测试提供了便利。我们尚未发现其他具备即时调用式强化学习API集成功能的开源OVE。

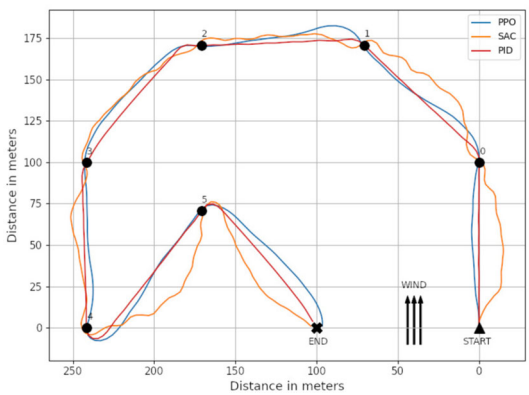



**图27** E艇的运动轨迹

在雅拉公司，我们研发了E-Boat——这款基于巴西制造的真实帆船机器人数字孪生体，专门用于环境监测。通过内置物理模型，可精准模拟其在波浪、水流及风力等不同环境条件下的运动轨迹。该模型采用标准帆板而非刚性帆板进行建模。

我们的实验表明，E-Boat平台模拟的帆船在不同风况下的操控性能与响应特性与真实世界行为高度吻合，充分展现了Yara系统中物理建模的精准度。通过与Gymnasium和Stable-baselines3模型的集成，我们成功实现了强化学习智能体在自主 GNC 决策方面的训练。第5.2.3节展示的实验结果表明，Yara作为可靠工具在实施、测试各类 GNC 策略及进行性能对比方面具有显著实用性。

在后续工作中，我们将通过建立波浪模型与风场之间的交互界面来改进该模型，从而使风场条件能够影响水表面状态。我们还将

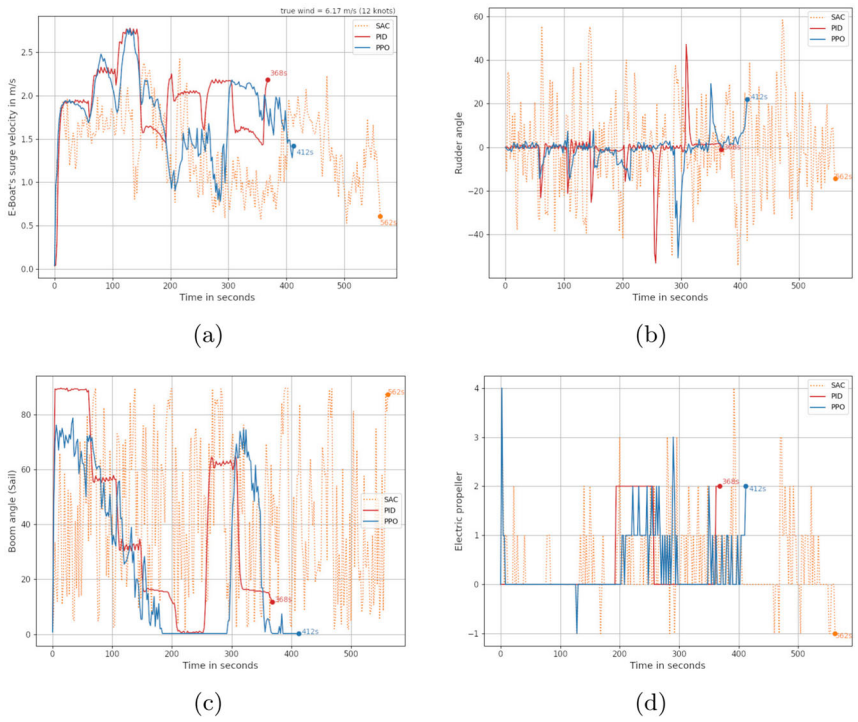



**Fig. 28** Mission data

include fog and rain conditions. We will also include fog and rain conditions. Other parameters of the PPO will also be worked on in the future implementations.

Concerning GNC's decision policies, the initial results were very promising. Suda & Nokoviski [22], Mexas et al. [21] and Rieppi et al. [31] presented some promising results using RL agents to control sailing boats. We will investigate new return functions that can improve the results we already have. Two challenging scenarios happen when the waypoint is in the "no go zone" (see Fig. 29) and in strong wind conditions. Suda & Nokoviski [22] tested PPO and SAC to learn optimized tack maneuver, but they used a 3-DOF simulation, and their approach must be checked in a more complex environment as Yara. We believe the answer is a hierarchical controller combining RL-trained policies with other classic types of controllers. We will test that hypothesis in future work.

Finally, the results obtained with the Yara OVE support its effectiveness and value as a tool for developing, training, and evaluating autonomous GNC systems for sailing robots on an RL-based approach. With further refinements and future research, Yara has the potential to contribute significantly to the advancement of autonomous sailing robot technology, ultimately leading to improved performance and capabilities in real-world applications.

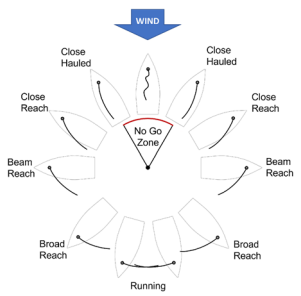



**Fig. 29** Sailing points

**Supplementary information** Our Yara OVE is available at https://github.com/medialab-fboat/Yara_OVE.git

**Acknowledgements** We acknowledge and be thankful to CAPES AMSUD (proj. 88881.522966/2020-01 NVIDIA, CNPq-FNDCT-MCTI (405535/2022-8) and the City of Niterói (Brazil) for supporting and funding this research.

**Author Contributions** All authors contributed to the conception, design, and evaluation of the Yara ocean virtual environment and the validation experiments. Yara was implemented by Eduardo Charles Vasconcellos with the contributions of Álvaro Pinto Fernandes de Negreiros, André Paulo Dantas de Araújo, and Raphael Guerra. The E-Boat maneuverability was evaluated by Álvaro Pinto Fernandes de Negreiros, Esteban Walter Gonzales Clua, and Luiz Marcos Garcia Gonçalves. Eduardo Charles Vasconcellos and Philippe Preux were responsible for the RL agent design and evaluation and for integrating the RL APIs into the environment. Eduardo Charles Vasconcellos, André Paulo Dantas de Araújo, and Philippe Preux wrote the first draft of the manuscript. All authors commented on previous versions of the manuscript. They read and approved the final manuscript.

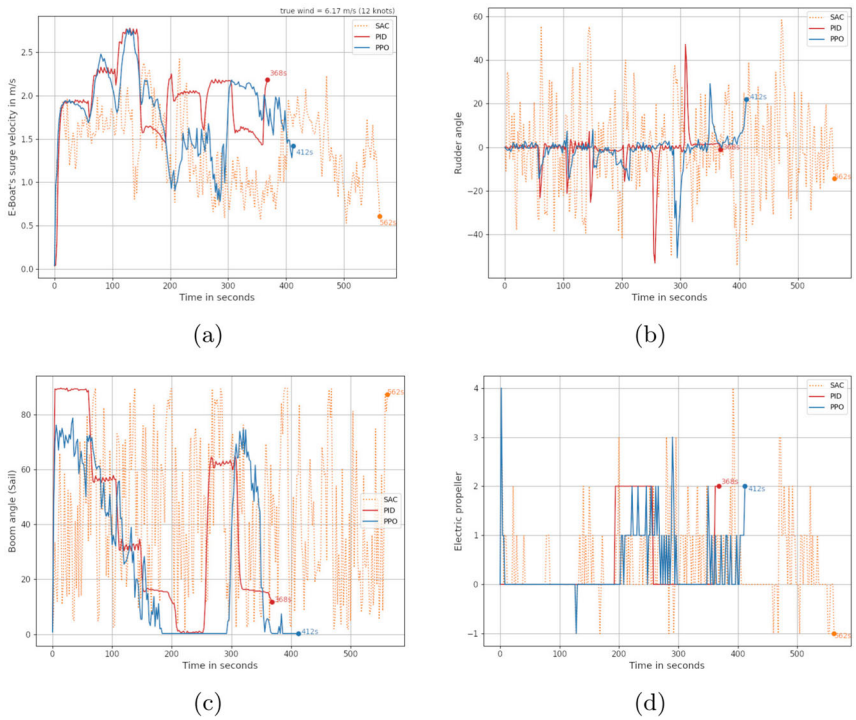



**图28**任务数据

包括雾天和雨天条件。我们还将纳入雾天和雨天条件。未来实施过程中还将对PPO的其他参数进行优化。

关于 GNC 决策策略，初期结果非常令人鼓舞。Suda与Nokoviski[22]、Mexas等学者的研究表明：[21] 以及Rieppi等[31] 我们利用强化学习智能体控制帆船时取得了一些令人鼓舞的成果。接下来我们将探索新型回归函数，以期进一步提升现有性能表现。当航点位于“禁航区”（见图29）或遭遇强风天气时，系统会面临双重挑战。Suda与Nokoviski[22]曾通过PPO和SAC算法进行优化转向机动学习测试，但其采用的3 DOF 仿真环境存在局限性，实际应用中需在Yara等复杂环境中验证方法有效性。我们认为解决方案在于将强化学习策略与传统控制器类型相结合的分层控制架构，该假设将在后续研究中进行验证。

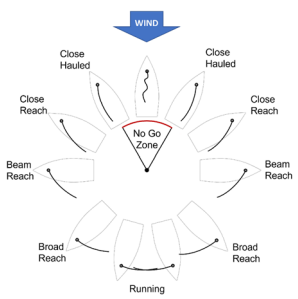



**图29**航行点

最终，Yara OVE实验结果证实了其作为基于强化学习方法开发、训练和评估航行机器人自主 GNC 系统工具的有效性与价值。通过进一步优化与未来研究，Yara有望显著推动自主航行机器人技术发展，最终提升实际应用中的性能与功能表现。

**补充信息** 我们的Yara OVE可在 https://github.com/medialab-fboat/Yara_OVE.git

**致谢** 我们衷心感谢Capes AMSUD（项目编号88881.522966/2020-01 NVIDIA 、CNPq- FNDCT - MCTI（405535/2022-8）以及巴西尼泰罗伊市对本研究的支持与资助。

**作者贡献**所有作者均参与了Yara海洋虚拟环境的概念设计、方案制定与评估工作，并共同完成了验证实验。Eduardo Charles Vasconcellos主导了Yara系统的实现开发，Alvaro Pinto Fernandes de Negreiros、Andre Paulo Dantas de Araujo及Raphael Guerra提供了技术支持。E-Boat机动性能测试由Alvaro Pinto Fernandes de Negreiros、Esteban Walter Gonzales Clua 和 Luiz Marcos Garcia Goncalves共同完成。Eduardo Charles Vasconcellos与Philippe Preux负责强化学习智能体的设计评估，并将强化学习API集成至系统环境。论文初稿由Eduardo Charles Vasconcellos、Andre Paulo Dantas de Araujo和Philippe Preux共同撰写。全体作者对论文初稿进行了审阅修改，并最终审定通过定稿版本。

**Funding** This work was supported by CAPES AMSUD (proj. 88881. 522966/2020-01 NVIDIA, CNPq-FNDCT-MCTI (405535/2022-8) and the City of Niterói (Brazil).

**Data Availability** The trained agent and the training data are available within Yara's code at https://github.com/medialab-fboat/Yara_OVE.git

## Declarations

**Conflicts of Interest** The authors have no relevant financial or non-financial interests to disclose.

**Ethical Approval** not applicable

**Consent to Participate** not applicable

**Consent for Publication** not applicable

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

## References

1. Theja, V.B., Yachameni, P., Shakeera, S., Venkataraman, H.: Integration of gazebo and ros for underwater vehicle environment. In: Proc. of OCEANS 2022 - Chennai, vol. 1, pp. 1–6 (2022)
2. Sun, Q., Qi, W., Liu, H., Ji, X., Qian, H.: Toward long-term sailing robots: State of the art from energy perspectives. Front. Robot. AI **8**, 416 (2022)
3. Negreiros, A.P.F., Correa, W.S., Araujo, A.P.D., Santos, D.H., Vilas Boas, J.M., Dias, D.H.N., Clua, E.W.G., Gonçalves, L.M.G.: Sustainable solutions for sea monitoring with robotic sailboats: N-boat and f-boat twins. Frontiers in Robotics and AI, 93 (2022)
4. Ang, Y.-T., Ng, W.-K., Chong, Y.-W., Wan, J., Chee, S.-Y., Firth, L.B.: An autonomous sailboat for environment monitoring. In: 2022 Thirteenth International Conference on Ubiquitous and Future Networks (ICUFN), pp. 242–246 (2022). IEEE
5. Mendonça, R., Santana, P., Marques, F., Lourenço, A., Silva, J.a., Barata, J.: Kelpie: A ros-based multi-robot simulator for water surface and aerial vehicles. In: 2013 IEEE International Conference on Systems, Man, and Cybernetics, pp. 3645–3650 (2013)
6. Plumet, F., Pêtres, C., Romero-Ramirez, M.-A., Gas, B., Ieng, S.-H.: Toward an autonomous sailing boat. IEEE J. Ocean. Eng. **40**(2), 397–407 (2015)
7. Jing, P., Hu, H., Zhan, F., Chen, Y., Shi, Y.: Agent-based simulation of autonomous vehicles: A systematic literature review. IEEE Access **8**, 79089–79103 (2020)
8. Lapan, M.: Deep Reinforcement Learning Hands-On: Apply Modern RL Methods to Practical Problems of Chatbots, Robotics, Discrete Optimization, Web Automation, and More. Packt Publishing Ltd, Birmingham, UK (2020)
9. Gazebo: Gazebo Web Page: Understand how to simulate underwater vehicles. (2024). https://gazebosim.org/api/gazebo/5.0/underwater_vehicles.html
10. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: Reliable reinforcement learning implementations. J. Mach. Learn. Res. **22**(1), 12348–12355 (2021)
11. Farama Foundation: Gymnasium. (2024). https://github.com/Farama-Foundation/Gymnasium
12. Plumet, F., Saoud, H., Hua, M.-D.: Line following for an autonomous sailboat using potential fields method. In: Proc. of 2013 MTS/IEEE OCEANS - Bergen, pp. 1–6 (2013)
13. Shen, Z., Wang, S., Yu, H., Guo, C.: Online speed optimization with feedforward of unmanned sailboat via extremum seeking without steady-state oscillation. Ocean Eng. **189**, 106393 (2019)
14. da Silva-Junior, A.G., Dos Santos, D.H., de Negreiros, A.P.F., Silva, J.M.V.B.S., Gonçalves, L.M.G.: High-level path planning for an autonomous sailboat robot using q-learning. Sensors (Switzerland) **20**(6) (2020)
15. Shen, Z., Ding, W., Liu, Y., Yu, H.: Path planning optimization for unmanned sailboat in complex marine environment. Ocean Eng. **269**, 113475 (2023)
16. Pêtrés, C., Romero-Ramirez, M.-A., Plumet, F.: Reactive path planning for autonomous sailboat. In: 2011 15th International Conference on Advanced Robotics (ICAR), pp. 112–117 (2011). IEEE
17. Zhou, L., Chen, K., Dong, H., Chi, S., Chen, Z.: An improved beetle swarm optimization algorithm for the intelligent navigation control of autonomous sailing robots. IEEE Access **9**, 5296–5311 (2021)
18. Liu, L., Wang, C., Gao, H., Shen, D., Liao, Y.: High-level path planning of unmanned sailboat for sailing championship and innovative education. In: 2022 IEEE International Conference on Unmanned Systems (ICUS), pp. 1557–1562 (2022)
19. Orozco-Rosas, U., Picos, K., Pantrigo, J.J., Montemayor, A.S., Cuesta-Infante, A.: Mobile robot path planning using a qapf learning algorithm for known and unknown environments. IEEE Access **10**, 84648–84663 (2022)
20. Tipsuwan, Y., Sanposh, P., Techajaroonjit, N.: Overview and control strategies of autonomous sailboats—a survey. Ocean Eng. **281**, 114879 (2023)
21. Méxas, R.P., Leta, F.R., Clua, E.W.G.: Comparison of reinforcement and imitation learning algorithms in autonomous sailboat digital twins. IEEE Lat. Am. Trans. **20**(9), 2153–2161 (2022)
22. Suda, T., Nikovski, D.: Deep reinforcement learning for optimal sailing upwind. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2022)
23. Liu, S., Yu, Z., Wang, T., Chen, Y., Zhang, Y., Cai, Y.: Mpc-based collaborative control of sail and rudder for unmanned sailboat. J. Mar. Sci. Eng. **11**, 460 (2023)
24. Saoud, H., Hua, M.D., Plumet, F., Amar, F.B.: Routing and course control of an autonomous sailboat. In: 2015 European Conference on Mobile Robots (ECMR), pp. 1–6 (2015)
25. Paravisi, M., Santos, D.H., Jorge, V., Heck, G., Gonçalves, L.M.G., Amory, A.: Unmanned surface vehicle simulator with realistic environmental disturbances. Sensors **19**(5) (2019)
26. Prats, M., Perez, J., Fernández, J.J., Sanz, P.J.: An open source tool for simulation and supervision of underwater intervention missions. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 2577–2582 (2012). IEEE
27. Manhães, M.M.M., Scherer, S.A., Voss, M., Douat, L.R., Rauschenbach, T.: Uuv simulator: A gazebo-based package for underwater intervention and multi-robot simulation. In: OCEANS 2016 MTS/IEEE Monterey, pp. 1–8 (2016). IEEE
28. Bingham, B., Aguero, C., McCarrin, M., Klamo, J., Malia, J., Allen, K., Lum, T., Rawson, M., Waqar, R.: Toward maritime robotic

**资金支持**本研究获得以下机构资助：Capes AMSUD（项目编号88881.522966/2020-01 NVIDIA 、CNPq- FNDCT - MCTI（405535/2022-8）以及巴西尼泰罗伊市政府。

**数据可用性**训练好的智能体和训练数据可在Yara代码库中获取，网址为https://github.com/medialab-fboat/Yara_OVE.git

## 声明

**利益冲突**作者无需披露的相关财务或非财务利益。

**伦理审批**不适用**参与研究知情同意**

不适用**发表论文知情同意**不适用

**开放获取**本文采用知识共享署名-非商业性使用-禁止演绎4.0国际许可协议授权，允许以任何形式进行非商业性使用、分享、传播及复制，但需满足以下条件：标明原始作者及来源信息、提供知识共享许可协议链接、注明是否对授权材料进行过修改。根据该许可协议，您无权分享本文衍生的改编材料或其部分内容。文中所含图片及其他第三方素材均受知识共享许可协议约束，除非在素材署名行另有说明。若素材未包含在知识共享许可协议范围内，且您的使用意图违反法律法规或超出许可范围，则需直接向版权方申请授权。如需查看许可协议副本，请访问http://creativecommons.org/licenses/by-nc-nd/4.0/。

## 参考文献

1. Theja，V.B.，Yachameni，P.，Shakeera，S.，Venkataraman，H.：集成遮阳棚与玫瑰结构用于水下航行器环境。载于：《2022年海洋学会议论文集——金奈》，第1卷，第1–6页（2022年）
2. 孙Q、齐W、刘H、季X、钱H：迈向长期航行机器人：从能源角度审视最新进展。Front. Robot. AI **8**，416（2022）
3. 内格雷罗斯，A.P.F.，科雷亚，W.S.，阿劳霍，A.P.D.，桑托斯，D.H.，维拉斯·博阿斯，J.M.，迪亚斯，D.H.N.，克鲁阿，E.W.G.，冈萨尔维斯，L.M.G.：机器人帆船实现海洋监测的可持续解决方案：Nboat与f-boat双体船。《机器人与人工智能前沿》，93（2022）
4. Ang，Y.-T.、Ng，W.-K.、Chong，Y.-W.、Wan，J.、Chee，S.-Y.、Firth，L.B.：一种用于环境监测的自主帆船。载于：2022年 Thirteenth International 泛在与 Future Networks 会议（ICUFN），第242–246页（2022年）。IEEE
5. Mendonca，R.、Santana，P.、Marques，F.、Lourenco，A.、Silva，J. a.，Barata，J.：Kelpie：一种基于ROS的水面及飞行器多机器人仿真系统。载于：2013年IEEE International Conference 系统、人与控制论会议，第3645–3650页（2013年）
6. Plumet，F.，Petres，C.，Romero-Ramirez，M.-A.，Gas，B.，Ieng，S.-H.：自主航行船的研发方向. IEEE海洋工程杂志**40**(2)，397–407 (2015)
7. Jing，P.，Hu，H.，Zhan，F.，Chen，Y.，Shi，Y.：基于Agent的自动驾驶车辆仿真：系统性文献综述。IEEE Access **8**，79089–79103（2020）
8. Lapan，M.：《深度强化学习实践指南：将现代强化学习方法应用于聊天机器人、机器人技术、离散优化、网络自动化等实际问题》。Packt Publishing Ltd，英国伯明翰（2020年）
9. Gazebo：Gazebo网页：了解如何模拟水下车辆。（2024年）。https://gazebosim.org/api/gazebo/5.0/underwater_vehicles.html
10. Raffin，A.，Hill，A.，Gleave，A.，Kanervisto，A.，Ernestus，M.，Dormann，N.：Stable-baselines3：可靠的强化学习实现方法. 机器学习研究杂志**22**(1)，12348–12355 (2021)
11. Farama基金会：体育馆。（2024年）。https://github.com/Farama-Foundation/Gymnasium
12. Plumet，F.，Saoud，H.，Hua，M.-D.：基于势场法的自主帆船航线跟踪研究。载于：2013年MTS/IEEE海洋会议论文集——卑尔根，第1–6页（2013年）
13. 沈Z.、王S.、余H.、郭C.：基于极值搜索法的无舵帆船在线速度优化与前馈控制（避免稳态振荡）。《海洋工程》**189**卷，106393页（2019年）
14. da Silva-Junior，A.G.，Dos Santos，D.H.，deNegreiros，A.P.F.，Silva，J.M.V.B.S.，Goncalves，L.M.G.：基于q学习的自主帆船机器人高级路径规划. Sensors（瑞士）**20**(6) (2020)
15. 沈振、丁伟、刘勇、余浩：复杂海洋环境中无人帆船路径规划优化。海洋工程**269**，113475（2023）
16. Petres，C.，Romero-Ramirez，M.-A.，Plumet，F.：自主帆船的反应式路径规划. 收录于: 2011年第15届 Advanced Robotics International Conference (ICAR)，第112–117页(2011). IEEE
17. 周立、陈科、董浩、池帅、陈志：一种改进的甲虫群优化算法用于自主航行机器人的智能导航控制。IEEE Access **9**，5296–5311（2021）
18. Liu，L.、Wang，C.、Gao，H.、Shen，D.、Liao，Y.：无人帆船高级路径规划技术在帆船锦标赛及创新教育中的应用。载于：2022 年 IEEE Unmanned Systems International Conference（ICUS），第1557–1562页（2022年）
19. Orozco-Rosas，U.，Picos，K.，Pantrigo，J.J.，Montemayor，A.S.，Cuesta-Infante，A.：基于QAPF学习算法的移动机器人路径规划方法（适用于已知与未知环境）。IEEE Access **10**，84648–84663（2022）
20. Tipsuwan，Y.，Sanposh，P.，Techajaroonjit，N.：自主帆船概述与控制策略——综述. 海洋工程**281**，114879 (2023)
21. Mexas，R.P.，Leta，F.R.，Clua，E.W.G.：自主帆船数字孪生系统中强化学习与模仿学习算法的比较. IEEE拉丁美洲汇刊**20**(9)，2153–2161 (2022)
22. Suda，T.，Nikovski，D.：基于深度强化学习的逆风航行优化策略。载于：2022年 International Joint Neural Networks 会议（IJCNN），第1–8页（2022年）
23. 刘S、余Z、王T、陈Y、张Y、蔡Y：基于MPC的无人帆船帆与舵协同控制研究。《海洋科学与工程杂志》**11**卷，460页（2023年）
24. Saoud，H.、Hua，M.D.、Plumet，F.、Amar，F.B.：自主帆船的航路规划与航向控制。载于：2015年 Mobile Robots European Conference（ECMR），第1–6页（2015年）
25. Paravisi，M.，Santos，D.H.，Jorge，V.，Heck，G.，Goncalves，L.M.G.，Amory，A.：带真实环境干扰的无人水面舰艇模拟器. Sensors **19**(5) (2019)
26. Prats，M.、Perez，J.、Fernandez，J.J.、Sanz，P.J.：一种用于水下干预任务模拟与监督的开源工具。收录于：2012年IEEE/RSJ Intelligent Robots 与系统 International Conference，第2577–2582页（2012年）。IEEE
27. Manhaes，M.M.M.，Scherer，S. A.，Voss，M.，Douat，L.R.，Rauschenbach，T.：Uuv模拟器：基于凉亭架构的水下干预与多机器人仿真软件包。载于：oceans 2016 MTS/IEEE Monterey会议论文集，第1–8页（2016年）。IEEE
28. Bingham，B.，Aguero，C.，McCarrin，M.，Klamo，J.，Malia，J.，Allen，K.，Lum，T.，Rawson，M.，Waqar，R.：海洋机器人技术研究进展